

AsTAR: Sustainable Energy Harvesting for the Internet of Things through Adaptive Task Scheduling

FAN YANG, Research Intelligent IoT and Computing X, Bosch, China and imec-DistriNet, KU Leuven, Belgium

ASHOK SAMRAJ THANGARAJAN, imec-DistriNet, KU Leuven, Belgium

GOWRI SANKAR RAMACHANDRAN, School of Computer Science, Queensland University of Technology

WOUTER JOOSEN and DANNY HUGHES, imec-DistriNet, KU Leuven, Belgium

Battery-free Internet-of-Things devices equipped with energy harvesting hold the promise of extended operational lifetime, reduced maintenance costs, and lower environmental impact. Despite this clear potential, it remains complex to develop applications that deliver sustainable operation in the face of variable energy availability and dynamic energy demands. This article aims to reduce this complexity by introducing AsTAR, an energy-aware task scheduler that automatically adapts task execution rates to match available environmental energy. AsTAR enables the developer to prioritize tasks based upon their importance, energy consumption, or a weighted combination thereof. In contrast to prior approaches, AsTAR is autonomous and self-adaptive, requiring no *a priori* modeling of the environment or hardware platforms. We evaluate AsTAR based on its capability to efficiently deliver sustainable operation for multiple tasks on heterogeneous platforms under dynamic environmental conditions. Our evaluation shows that (1) comparing to conventional approaches, AsTAR guarantees *Sustainability* by maintaining a user-defined optimum level of charge, and (2) AsTAR reacts quickly to environmental and platform changes, and achieves *Efficiency* by allocating all the surplus resources following the developer-specified task priorities. (3) Last, the benefits of AsTAR are achieved with minimal performance overhead in terms of memory, computation, and energy.

CCS Concepts: • **Computer systems organization** → **Sensor networks**; • **Hardware** → *Sensor devices and platforms*; *Wireless integrated network sensors*; • **Software and its engineering** → *Real-time systems software*;

Additional Key Words and Phrases: Energy harvesting, adaptive scheduling, internet of things

ACM Reference format:

Fan Yang, Ashok Samraj Thangarajan, Gowri Sankar Ramachandran, Wouter Joosen, and Danny Hughes. 2021. AsTAR: Sustainable Energy Harvesting for the Internet of Things through Adaptive Task Scheduling. *ACM Trans. Sen. Netw.* 18, 1, Article 4 (October 2021), 34 pages.
<https://doi.org/10.1145/3467894>

Extended version of EWSN19 paper “AsTAR: Sustainable Battery Free Energy Harvesting for Heterogeneous Platforms and Dynamic Environment”.

This research is partially funded by the Research Fund KU Leuven.

Authors' addresses: F. Yang, Research Intelligent IoT and Computing X, Bosch, 333 Fuquan N Rd, Changning District, Shanghai, China and imec-DistriNet, KU Leuven, Belgium; email: fan.yang@kuleuven.be; A. S. Thangarajan, W. Joosen and D. Hughes, imec-DistriNet, Celestijnenlaan 200A, Leuven, Belgium; emails: ashoksamraj.thangarajan@kuleuven.be, wouter.joosen@kuleuven.be, danny.hughes@kuleuven.be; G. S. Ramachandran, School of Computer Science, Queensland University of Technology, 2 George St Brisbane, QLD 4000, Australia; email: g.ramachandran@qut.edu.au.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2021 Association for Computing Machinery.

1550-4859/2021/10-ART4 \$15.00

<https://doi.org/10.1145/3467894>

1 INTRODUCTION

The **Internet of Things (IoT)** is being deployed at massive scale in homes, smart cities, and industrial applications. Analysts such as Gartner [18] and Forrester [20] predict that billions of IoT devices will be deployed within the next few years. In this context, the typical battery life of IoT devices, which ranges from a few years to a decade [19], is problematic. Manually replacing billions of batteries is not economically feasible, and leaving large quantities of toxic batteries embedded in our environment is not sustainable, as batteries require many years to fully decompose. Energy harvesting in combination with super-capacitors for charge storage is a promising avenue to move beyond the operational lifetime offered by conventional batteries and thereby tackle these problems. By ensuring that IoT devices can be perpetually powered by light, thermal or kinetic energy harvested from the environment, batteries can be eliminated, conserving both human effort and the environment.

While the potential of energy harvesting in the IoT is clear, it is notoriously complex to develop reliable energy-harvesting applications due to three factors: (i) Environmental dynamism, which gives rise to an unpredictable energy supply, (ii) heterogeneity in platform and peripheral power demands, and (iii) the *tragedy of the coulombs* [10] wherein a single energy-hungry software module can starve all other modules of energy, rendering the device inoperable.

This article addresses these three problems by introducing the **Asymmetric Task Adaptation Rate scheduler (AsTAR)**, a self-adaptive task scheduler for building energy-harvesting IoT applications. AsTAR addresses environmental dynamism by providing *self-adaptive task scheduling*. Sustainability is ensured by throttling task execution rates as required to acquire and sustain a developer-specified optimal level of charge. AsTAR is fully autonomous and requires no pre-configuration or *a priori* modeling to deal with different energy-harvesting sources or peripheral hardware, which may even be modified at runtime. Fine-grained control over task execution and peripheral usage prevent a single energy-hungry software module from rendering the system non-functional. AsTAR fairly schedules tasks based on user-defined priorities, their energy consumption or a weighted combination thereof. The novel contributions of AsTAR are threefold: (i) Simple, yet effective energy-aware task scheduling on IETF Class-1 devices [14], (ii) Support for platform heterogeneity and dynamism with zero up-front modeling, and (iii) A reference platform for experimentation with sustainable energy-harvesting applications.

We evaluate AsTAR using a combination of simulation, laboratory tests and a realistic deployment scenario, using indoor solar energy harvesting. The evaluation shows that (i) AsTAR achieves its goal of amassing an optimal charge level on heterogeneous hardware platforms. (ii) AsTAR adapts quickly to dynamic levels of power generation or consumption. (iii) AsTAR fairly schedules tasks according to developer-specified priorities or the energy impact of those tasks. (iv) AsTAR has extremely low performance overhead even on IETF Class-1 devices. Considered in sum, we believe that AsTAR's features make a valuable contribution to simplifying the development of sustainable, multi-tasking, energy-harvesting IoT applications.

The remainder of this article is structured as follows. Section 2 discusses related work and highlights the gap that AsTAR addresses in the research literature. Section 3 presents the AsTAR approach including the adaptive algorithm and task scheduler, software stack, and hardware platform. Section 4 describes our reference implementation. Section 5 evaluates the performance of AsTAR. Finally, Section 6 concludes and discusses directions for future work.

2 RELATED WORK

We review three key streams of research that are related to energy harvesting. Section 2.1 reviews prior work on energy-harvesting and battery-free platforms. Section 2.2 discusses OS support for

energy harvesting. Section 2.3 discusses software development support for energy-harvesting applications. Finally, in Section 2.4, we highlight the gap in prior research that is addressed by AsTAR. For a more extensive review of prior energy-harvesting research, we refer the reader to Reference [1].

2.1 Energy-harvesting IoT Platforms

The first generation of IoT energy-harvesting platforms augmented battery-powered systems with solar panels to extend the service-life of devices and to provide an expanded energy envelope for system operation. Raghunathan et al. introduced Heliomote [28], a energy-harvesting add-on board for the popular Mica2 mote. It is able to fully charge a battery in a diurnal scenario, achieving a near-perpetual, energy-aware operation. The follow-up work [25] manages the harvested energy and allocate workload accordingly, by applying an energy prediction model on a slotted historic average. It achieved a finer control of the workload with the prediction of available energy, but the system parameter needs to be calibrated in the environment prior deployment. Hughes et al. developed GridStix [13], a wireless sensor network for flood monitoring and warning that was capable of sustainable multi-year operation using a combination of solar panels and rechargeable batteries. Taneja et al. [26] and Buchli et al. [2] contribute systematic models of solar energy production in outdoor sensing scenarios. Hsu et al. [12] tackle the problem of optimally scheduling solar device operation under variable solar energy supply based upon a combination of in-situ benchmarking and modeling. Voigt et al. introduced solar-aware routing in Reference [29], where packets are preferably routed through nodes that are solar-powered. While the majority of work in the area of predictive energy harvesting has focused on solar power, Gaglione et al. also demonstrated sustainable battery-free operation using harvested kinetic energy in a vibration-based structural health monitoring application [9] using a combination of benchmarking and simulation.

Contemporary energy-harvesting platforms enable a broad range of energy sources such as vibration, thermal and solar, and eliminate batteries as energy storage to prolong the operational life of IoT devices. While the shelf life of an appropriately selected super-capacitor is orders of magnitude longer than a standard battery, the charge density of capacitors is far lower than that of batteries. The result is that battery-free sensing platforms have a much smaller and more dynamic energy envelope than battery-powered systems. Hester et al., identify the “*Tragedy of the Coulombs*” in energy-harvesting systems wherein a single energy-hungry hardware or software component can consume sufficient energy to render the entire system inoperable. **United Federation of Peripherals (UFoP)** [10] addresses this problem through a hardware-based solution, wherein harvested energy is divided into multiple peripheral-specific capacitors, which provide independent power sources and therefore prevent a single sub-optimal peripheral from causing the Tragedy of the Coulombs. Flicker [11] builds on UFoP, extending the concept of isolated peripheral energy storage to support plug-and-play peripheral boards that may be flexibly connected at development time. Hester et al. demonstrate through representative user-trials that the isolation of peripheral power consumption significantly reduces the difficulty of realizing energy-harvesting applications.

The approaches to energy harvesting described above fall into two broad categories. The first category of research is to provide *sustainable* operation in the face of dynamic energy availability [13, 26]. The second approach embraces *intermittent* operation [10, 11]. AsTAR contributes to the sustainability stream of research, aiming to ensure sustainable system operation on a battery-free energy-harvesting platform that can thereby function as a drop-in replacement for long-life battery powered systems.

2.2 OS Support for Energy Management

IoT **Operating Systems (OSs)** play a key role in managing the complexity of a dynamic energy envelope through the adaptive scheduling of tasks and the reconfiguration of system-level functionality. Nano-RK [8] provides energy awareness on an embedded **Real-time OS (RTOS)**. As with many Real-time Operating Systems, Nano-RK guarantees that task deadlines are met using priority-based preemptive multi-threading. Nano-RK mediates access to every system resource with a consistent API and is thereby capable of performing fine-grained energy accounting based upon an energy model, which is established prior to deployment. Nano-RK is deeply dependent upon its static energy model, which makes it difficult to apply on reconfigurable IoT platforms such as Flicker [11] and μ PnP [31].

Yildirim et al. contribute InK [32], a reactive kernel for battery-free IoT devices. InK targets devices with small energy storage reserves that operate intermittently when environmental energy becomes available. InK provides support for programming these kinds of intermittently operational devices based upon an event-driven and energy-aware execution model. In addition to a tailored programming paradigm, InK also offers dedicated support for managing time and memory in the face of intermittent operation. While these features make it significantly easier to build robust intermittent applications, the goals of InK are quite different from those of AsTAR, which aims to preserve sustainable operation rather than manage the problems of intermittent operation.

The Cinder OS [22] is built for energy sensitive mobile devices such as smartphones. Cinder supports energy management by providing developers with two dedicated abstractions, *reserves* and *taps*. A *reserve* permits an application to use a certain amount of energy, while a *tap* limits the maximum rate of energy that can be transferred or consumed. Cinder tasks may allocate energy from their *reserve* to sub-tasks and control the rate of energy consumption through the *tap*. These features together lower the burden on developers to create energy-aware applications. However, Cinder does not consider energy harvesting. Furthermore, it targets far more powerful mobile-phone class devices.

Cao et al. [3] provide support for energy management on multi-application IoT devices by introducing dedicated energy management abstractions in the form of *reserves*, which provide a virtual representation of the amount of energy available to an application. Isolation of energy reserves is enforced by the system at runtime to ensure fair access to energy resources on shared platforms. When an application exhausts its reserve, it is terminated. This approach is dependent upon a common virtual machine that mediates and can therefore control access to low-level, energy-consuming resources.

Eon [24] is a programming language and supporting runtime that is designed to facilitate the development of perpetually powered systems. Eon allows programmers to define multiple paths or *flows* through a program. The Eon scheduler then selects flows and sets their rates of execution based upon the current energy levels to maintain sustainable operation. Eon makes energy supply predictions based upon an **Exponentially Weighted Moving Average (EWMA)** of observed energy generation, while energy consumption estimates are based upon benchmarks obtained from testing of each hardware and software platform.

2.3 Software Development for Energy-harvesting Applications

Prior work has contributed a number of software engineering tools to reduce the complexity of developing sustainable energy-harvesting applications.

Eco [33] is a programming model for sustainable software. Programs written in Eco may adaptively adjust their behavior to stay within a given energy or temperature budget. Eco achieves a fine-grained, programmable, and declarative concept of sustainability by the language runtime

consistently matching energy supply and demand. This feature would significantly simplify the creation of sustainable energy-harvesting applications. However, the Eco runtime targets only server-class devices with orders of magnitude more resources than typical IoT devices.

Ritter et al. [21] combine software instrumentation and super-capacitor charge storage to accurately estimate the lifetime of an IoT device. By replacing the battery of the device with an appropriately sized capacitor and measuring the voltage before and after the execution of a function, Ritter et al. demonstrate that it is possible to accurately measure energy consumption based upon the near linear relationship between the capacitor voltage and average current with a fixed capacitor. While the authors do not focus on energy harvesting, the article showcases a secondary benefit of using super-capacitors beyond their long lifespan; the ability to profile energy consumption in the field. During a relatively short execution time, the voltage drop can be presented as

$$\delta V = \frac{I_{LKG} + I_{load}}{C},$$

where I_{LKG} is the leakage current of the device, I_{load} is the load current, and C is the capacitance. In practical cases when I_{load} is more than one thousand times larger than I_{LKG} , one could calculate the load current from the voltage drop.

The proposed methodology is dependent upon manual code instrumentation and therefore should be expected to incur significant development-time overhead.

Lachenmann et al. [16] append energy consumption data to software tasks and enable developers to assign tasks within energy *levels*, which may be deactivated to reduce energy consumption. The Levels task scheduler then selects the highest energy level, and thus group of associated tasks, which may be executed while still achieving a given lifetime goal. This approach ensures that battery lifetime goals are met, while allowing developers to prioritize tasks for execution. All energy estimates must be generated before deployment.

2.4 Gap Analysis and Requirements

We now reflect on the gap that AsTAR addresses with respect to the related work discussed in Sections 2.1–2.3.

- *Reducing the burden of benchmarking:* Many of the approaches described above [2, 16, 21, 26] demand the creation of detailed hardware models. Modeling requires significant effort and access to laboratory equipment, forming a barrier to entry for energy aware software development. We argue that energy management software should not require up-front benchmarking and instead gather all necessary information automatically at runtime.
- *Operation on heterogeneous platforms:* IoT applications increasingly run across multiple platforms and, for most energy management approaches, this requires extensive re-parameterization of an underlying energy model. The problem is even more acute for reconfigurable IoT platforms [11, 31], where the hardware base may change at runtime through the connection of new peripherals. Energy aware software should cope with platform heterogeneity without the need for per-platform developer intervention.
- *Flexible Multi-Tasking:* Prior work has provided a range of mechanisms to fairly schedule task execution based either upon a developer-specified priority or the energy consumption of individual tasks [16, 21, 33]. In our view, these notions should ideally be combined into a simple, yet expressive task prioritization scheme that allows developers to reason over both the *importance* and *energy cost* of tasks.
- *Mitigating dynamism:* To maximize applicability, energy management approaches for energy-harvesting IoT platforms should ideally respect the inherently unpredictable nature of many environments, where available energy from light, heat or vibration may vary

widely or even be lost without notice. In such unpredictable environments, long term modeling [2, 26] is likely to prove unproductive. In the case of reconfigurable IoT platforms such as Flicker [11] and μ PnP [31], the modification of connected peripherals at runtime also gives rise to dynamic power demands.

- *Low runtime overhead:* Finally, any energy management approach must naturally itself be efficient in terms of memory and computation if it is to be implemented on IETF Class-1 IoT devices. This precludes heavyweight techniques such as those from mainstream computer systems [22, 23, 33].

The following section discusses how the design of AsTAR achieves sustainable energy-harvesting operation, while addressing the gaps identified in prior research.

3 ASTAR APPROACH

AsTAR's energy aware adaptation adapts how frequent sensor tasks may execute. The more frequently a sensing task is executed, the higher the temporal resolution of sensed data. However, this higher resolution comes at the expense of greater energy consumption. AsTAR strives to balance these conflicting optimization goals by maximizing task execution to the greatest extent possible, while also ensuring that charge is reliably accumulated and sustained at a developer-specified optimum.

AsTAR aims to operate without lab-based benchmarking or environmental modeling to minimize developer effort. Instead, AsTAR continually observes and reacts to system state at runtime. AsTAR is designed to deal with platform *heterogeneity* in terms of: device power supply, task power demands and energy storage capacity. Furthermore, AsTAR aims to operate in unpredictable energy-harvesting environments with inconsistent task load, where it mitigates *dynamicism*. It is important to note that the AsTAR approach is limited in its applicability to tasks whose schedule can be independently controlled, such as sensing operations. AsTAR cannot schedule time-sensitive tasks due to the non-deterministic nature of its adaptive scheduler. However, time-sensitive tasks such as a transmit or receive slots in synchronized network protocol may run alongside AsTAR under a separate scheduler without disturbing AsTAR's operation. In fact, AsTAR will detect this extra load and reduce the rate at which AsTAR tasks are scheduled to compensate.

Section 3.1 motivates our selection of self adaptive algorithm. Section 3.2 describes the adaptive AsTAR task scheduling algorithm and how we have modified this algorithm for our application case. Section 3.3 describes the scheduling algorithm and discusses how AsTAR supports flexible multi-tasking. Section 3.4 then describes the AsTAR software platform. Finally, Section 3.5 describes the AsTAR hardware platform.

3.1 Selecting a Self-adaptive Algorithm

We focus on closed loop control algorithms as they are inherently simple and efficient to execute on embedded devices. In this class of algorithm, the controller monitors a feedback variable (capacitor voltage level), based upon which it configures a control element (task execution rate) in such a way as to prevent the feedback variable from deviating from its optimum value. Specifically, we draw inspiration from the TCP congestion control algorithm, which applies a variant of **Additive Increase/Multiplicative Decrease (AIMD)** [4] to control the rate at which hosts transmit packets into the network.

The problems of charge control and congestion control are intuitively similar. While TCP avoids congestion by adapting its congestion window based upon the signal of packet loss, AsTAR maximizes task execution rates while avoiding charge exhaustion by modifying task scheduling rates

based upon rising or falling charge levels. Beyond this intuitive link, Chiu et al. [5] highlight a number of positive features of AIMD that make it suitable for application in our scenario. Specifically it is: efficient, fair and converges to an equilibrium state even without global knowledge.

- *Efficiency* is defined as selecting a task execution rate that is close to its optimal value (i.e., the maximum rate that can be achieved without reducing the charge level).
- *Fairness* is achieved when equivalent tasks are assigned an equal share of energy resources (i.e., they are assigned the same task rate).
- *Operation on incomplete knowledge* is essential when the controller does not have a global overview of state. In the case of TCP, knowledge is incomplete as hosts compete with flows elsewhere on the network. In the case of AsTAR, knowledge is incomplete as low-level system libraries (e.g., a real-time network stack) are not managed by the scheduler.
- *Convergence* is the ability of the controller to quickly find the equilibrium state where it remains with minimal oscillation. The ideal controller is responsive, quickly reaching the equilibrium state and smooth, demonstrating minimal oscillation once this state is reached.

3.2 Adaptive Task Scheduling

The self-adaptive algorithm determines the task execution rates and it is embedded inside the AsTAR scheduler. AsTAR scheduler controls the AsTAR tasks, and the tasks must therefore be independently schedule-able. Examples of such tasks include: sending sensor telemetry, broadcasting location beacons or de-fragmenting flash storage.

AsTAR tracks charge levels by periodically measuring the capacitor voltage and, based upon the observed amount of stored charge, adapts task execution rates. AsTAR monitors and reacts to changes in stored charge, rather than energy supply as this inherently takes into account the complex interactions that occur between supply current, supply voltage, capacitor voltage and capacitor leakage current, all of which would otherwise need to be explicitly modeled. This section describes how the simple, yet effective AsTAR algorithm schedules task execution rates. Section 3.3 then discusses how multiple tasks are scheduled with respect to their energy impact and user-defined priority.

AsTAR defines two configurable voltage thresholds and a hysteresis. The system behavior is separated into four states based on the voltage of the super-capacitor, these states guide the operation of the task scheduling algorithm. The voltage thresholds may be configured by the developer prior to deployment and reconfigured at runtime. The four operation states are introduced below:

- **Shut-off Voltage state:** In this state, system voltage has fallen to a level below the user specified **Shut-off Voltage (SV)**. The primary goal of the system is to maximize sustainability by reducing power consumption. All application tasks are therefore disabled until sufficient charge accumulates to run all system components.
- **Low-voltage state:** In this state, which lies between the SV and **Optimum-voltage (OV)** thresholds, the goal of the system is to increase the frequency of tasks while ensuring that charge accumulates during each *task adaptation period*.

$$R(t+1) = \begin{cases} R(t) + a & \text{if capacitor is accumulating,} \\ R(t)/b & \text{if capacitor is not accumulating.} \end{cases}$$

AIMD is applied to schedule the task rate in this state. As shown in the equation above, the system gently probes the available energy supply by increasing the task rate R additively and, if charge ceases to accumulate in the capacitor, backs off aggressively by using multiplicative decrease to protect the charging process.

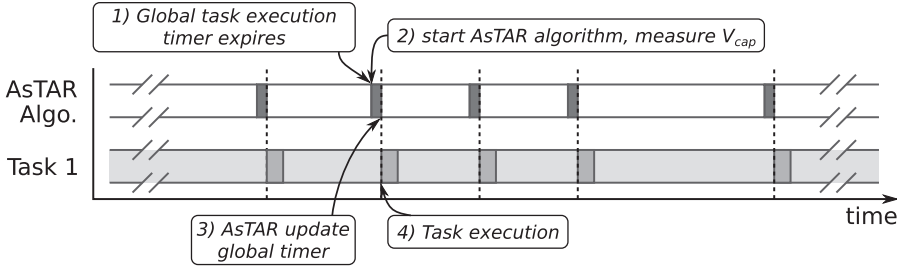


Fig. 1. The AsSTAR adaptive scheduler with a single task. The task execution rate/period changes according to the AsSTAR algorithm.

- **Optimum-voltage state:** In this state, charge levels are optimal and the goal of AsSTAR is to remain in this state. Task rates will not be adapted unless charge falls into the *Low-voltage state*, or rises into the *High Voltage state*. In this state, a variable hysteresis is applied to increase stability. By default, AsSTAR will not react to changes of less than 10 mV.
- **High-voltage state:** In this state, system voltage has risen above the *OV* threshold. The goal of the system is therefore to maximize the utilization of energy from the harvester.

$$R(t+1) = \begin{cases} R(t) - a & \text{if capacitor is not accumulating,} \\ R(t) \times b & \text{if capacitor is accumulating.} \end{cases}$$

AsSTAR applies AIMD in reverse: if charge does not decrease towards the optimum, tasks rates are **Multiplicatively Increased (MI)** to fully utilize the additional energy. Conversely, when voltage falls, AsSTAR gently reduces task rates with **Additive Decrease (AD)** to prevent under-shooting the optimum.

It should be noted that the developer is free to define *OV* and *SV* to whatever level is appropriate for their application. In selecting an appropriate value for *OV*, the developer should select a voltage that: maximizes energy efficiency, ensures long capacitor service life and provides an appropriate period of autonomous operation when energy harvesting fails. In the case of *SV*, the developer should consider the minimum *safe* operating voltage for the system. In our experience, these values can easily be derived from the component data sheet. While the optimal values for these parameters will vary across platforms, we believe that appropriate values may be similarly deduced for most battery-free energy-harvesting platforms.

The AsSTAR scheduling algorithm is simple and compact, as is shown by the pseudo code provided in Algorithm 1. The algorithm decides the operation of state by comparing the capacitor voltage level to voltage thresholds. In shut-off voltage state, where the voltage is too low to safely perform any tasks, the task rate is simply assigned to 0. In low-voltage state and high voltage state, the algorithm performs the corresponding behavior (AIMD/MIAD) based on whether the available energy is accumulating, or decreasing. In optimum voltage state, the task rate does not change.

Figure 1 shows a time diagram of AsSTAR's scheduler operation with a single task. The core of AsSTAR Task Scheduler is the *Global Execution Timer*, which operates as follows: (1) When the *Global Execution Timer* expires, (2) the scheduler measures the voltage in the capacitor and calls the *AsSTAR Algorithm*. (3) AsSTAR returns an updated global task execution rate, and the period of that execution rate, as the new *Global Execution Timer* period. (4) the task is executed. Once the execution cycle is complete, the system returns to sleep and waits for the next *Global Execution Timer* expiration. This is used to derive all AsSTAR schedules.

ALGORITHM 1: AsTAR algorithm for choosing next task execution rate.

Input : V_{new} , Current capacitor Voltage
 V_{old} , capacitor voltage from previous cycle
 R_{old} , previous global task rate and
hysteresis for increased stability.

Output: Updated global task rate R_{new}
Updated Global Execution Timer period GET_{new}

```

1 Function get_adaptation_rate( $V_{new}$ ,  $V_{old}$ ,  $R_{old}$ ) is
2   if  $V_{new} < Shut\_Off\_Voltage$  then                                     */
3     /* The system is in Shut-off Voltage state, task is stopped */
4      $R_{new} \leftarrow 0$ ;
5   end
6   else if  $V_{new} < (Optimum\_Voltage - hysteresis)$  then                 */
7     /* The system is in Low-voltage state, apply AIMD */
8     if  $V_{new} > V_{old}$  then
9        $R_{new} \leftarrow R_{old} + 1$ ;
10    end
11    else
12       $R_{new} \leftarrow R_{old}/2$ ;
13    end
14  end
15  else if  $V_{new} > (Optimum\_Voltage + hysteresis)$  then                 */
16    /* The system is in High Voltage state, apply MIAD */
17    if  $V_{new} < V_{old}$  then
18       $R_{new} \leftarrow R_{old} - 1$ ;
19    end
20    else
21       $R_{new} \leftarrow R_{old} \times 2$ ;
22    end
23  end
24  else
25    /* The system is at Optimum Voltage state, take no action. */
26     $R_{new} \leftarrow R_{old}$ ;
27  end
28   $GET_{new} \leftarrow 1/R_{new}$ ;
29  return  $V_{new}, R_{new}, GET_{new}$ ;
30 end

```

3.3 Multi-task Scheduling

AsTAR is designed to support multiple concurrent application tasks, to which it assigns a schedule that respects developer-specified task priorities, while ensuring that the sum of all tasks schedules balances energy supply and demand to keep the platform at an optimum level of charge. Multi-tasking support in AsTAR offers three modes of operation: (i) *round-robin scheduling*, (ii) *energy-aware scheduling*, and (iii) *prioritized scheduling*.

Round-robin scheduling: Evenly distributes available platform resources among different tasks. All tasks are treated equally, and AsTAR schedules tasks at the same rate. The individual

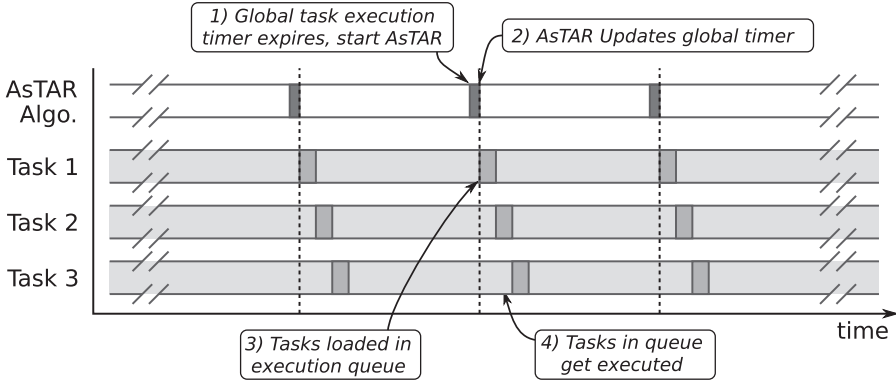


Fig. 2. Task scheduler with multiple unprioritized tasks using Round-robin scheduling.

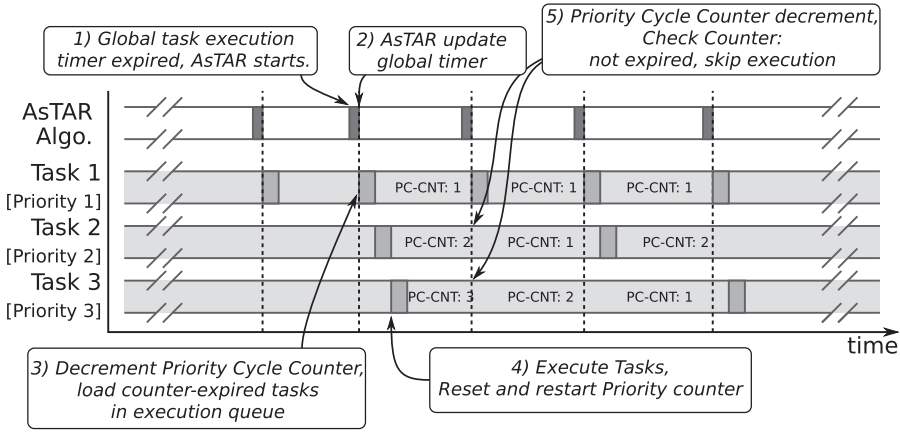


Fig. 3. Task scheduler scheduling multiple tasks with priority.

energy consumption and priority of tasks is not considered in scheduling, only global aggregate energy consumption.

An example is shown in Figure 2, where three tasks are registered in the system. Similar to single-task scheduling, (1) when the *Global Execution Timer* expires, the scheduler starts the *AsSTAR Algorithm*, and (2) AsSTAR returns the updated global task execution timer period. (3) Multiple tasks will be loaded into the *Task Execution Queue*, and (4) the tasks are executed. After the execution cycle is complete, the system goes to sleep and waits for the next *Global Execution Timer* expiration. As can be seen from the example shown in Figure 2, when using Round-robin scheduling, AsSTAR schedules all tasks equally. In other words, when the number of tasks increases along with the energy load on the system, all tasks will be scheduled at a slower, equal rate.

Prioritized scheduling: AsSTAR supports developer-specified task priorities, which allows application developers to divide system resources amongst tasks according to their importance. In this mode, tasks with higher priority will be executed more frequently, and vice-versa. Task priorities are initially set at deployment time, and may be reconfigured at runtime based upon changing application requirements.

Priority is realized with a *Priority Cycle Counter*. The *Priority Cycle Counter* acts as a *Global Execution Timer* cycle multiplier and tasks will only be executed when their cycle counter expires.

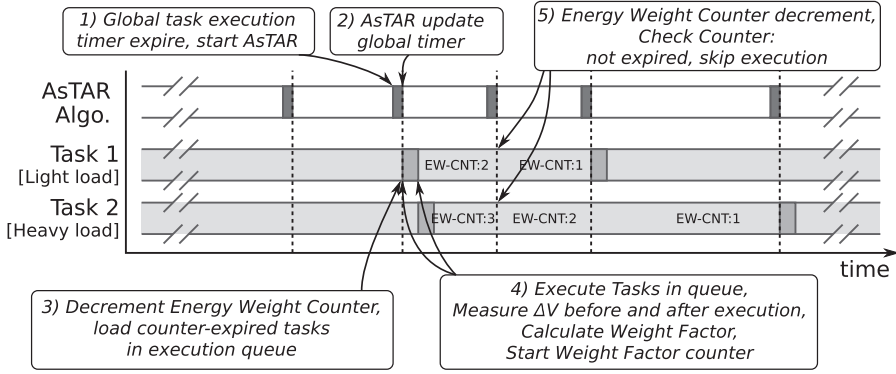


Fig. 4. Task scheduler: multiple tasks with weight factor.

An example of this is shown in Figure 3, where three tasks with different priorities (1, 2, and 3) are registered in the system. (1) When the *Global Execution Timer* expires, the scheduler starts the *AsTAR Algorithm* and (2) AsTAR returns an updated global task execution timer period. (3) *Priority Cycle Counters* of all tasks are decremented, tasks whose *Priority Cycle Counter* expires are loaded into the *Task Execution Queue*, and (4) the tasks are executed. (5) Upon the expiration of the *Global Execution Timer*, the process repeats. As the *Priority Cycle Counter* of Tasks 2 and 3 has not expired, these tasks are not loaded into the queue.

Energy-weighted scheduling: In the previous two scheduling modes, one power-hungry task will dramatically slow down all other tasks in the system, which may be undesirable for some applications. In energy-weighted scheduling mode, the energy impact of each task execution is measured in-situ, and the schedule of tasks is modified such that each task is allocated a roughly equal amount of energy. This throttles energy hungry tasks, to prevent them slowing down the performance of energy efficient tasks.

At runtime, AsTAR benchmarks the impact of task execution on stored energy reserves by measuring the change in capacitor voltage (δV) before and after task execution. This is used to generate per-task *Energy Weight Factors*, which are used by the *Energy Weight Counter*. A weight factor of 1 maps to the minimum measurable δV on the capacitor for a given platform, and a weight factor of 2 indicates double this δV and so on. For a task to have a weight factor below 1, its energy consumption would be so low as to be immeasurable. In this case, the task will be assigned weight factor 1. *Energy Weight Counter* works similarly to the *Priority Cycle Counter*; every time the *Global Execution Timer* expires, AsTAR decrements the *Energy Weight Counter*, executing the associated task only when the *Energy Weight Counter* expires. As with prioritized scheduling, tasks with higher *Energy Weight Factor* will be scheduled less frequently.

An example is shown in Figure 4, where two tasks with very different loads are registered in the system. (1) When the *Global Execution Timer* expires, the scheduler starts the *AsTAR Algorithm*, and (2) AsTAR returns the updated *Global Execution Timer* period. (3) All tasks whose *Energy Weight Counter* has expired are loaded into the *Task Execution Queue*. (4) Immediately before and after the task execution, AsTAR takes two samples of voltage in the capacitor, and the *Energy Weight Factor* is calculated and updated using ΔV . The *Energy Weight Counter* will then restart with an updated *Energy Weight Factor* as its initial value. (5) When next time the *Global Execution Timer* expires, the same operation repeats and all tasks with an expired *Energy Weight Counter* are executed.

Discussion: The three scheduling mechanisms we describe in this section co-exist in AsTAR Scheduler, wherein each task maintains its own *Task Cycle Counter*. Developers may use priorities together with energy weightings, which will be combined and interpreted by the scheduler as follows:

$$\text{Task Cycle Counter} = \text{Priority} \times \text{Energy Weight Factor}.$$

We have made the deliberate design decision to use coarse-grained *Energy Weight Factors*, which combine the dynamism of energy generation, task energy consumption, and also the size of capacitor. This ensures simplicity for the developer, in comparison to a fine-grained or continuous priority scheme.

ALGORITHM 2: AsTAR Scheduler logic

```

1  foreach task ∈ task_list do
2      task.cycle_counter ← task.cycle_counter − 1;
3      if task.cycle_counter = 0 then
4          if task.state = blocked then
5              // task is blocked by external event, do not execute
6          end
7          else if task.state = ready then
8              enqueue_to_execution_queue(task);
9              // task is inserted in the execution queue
10         end
11     end
12 Vold ← Vnew;
13 Vnew ← readADC(VCC);
14 Rnew ← get_adaptation_rate(Vnew, Vold, Rold);
15 // call AsTAR adaptation algorithm
16 Rold ← Rnew;
17 foreach task ∈ execution_queue do
18     dequeue_from_execution_queue(task);
19     // task is taken out of the execution queue
20     V1 ← readADC(VCC);
21     task.state ← running;
22     task.run();
23     // task is executed
24     V2 ← readADC(VCC);
25     // voltage is measured right before and after task execution
26     task.state ← ready;
27     task.energy_weight ← calculate_energy_weight(V1, V2);
28     // new energy weight factor is updated
29     task.priority ← user_defined_priority;
30     task.cycle_counter ← task.priority × task.energy_weight;
31 end
32 Sleep();

```

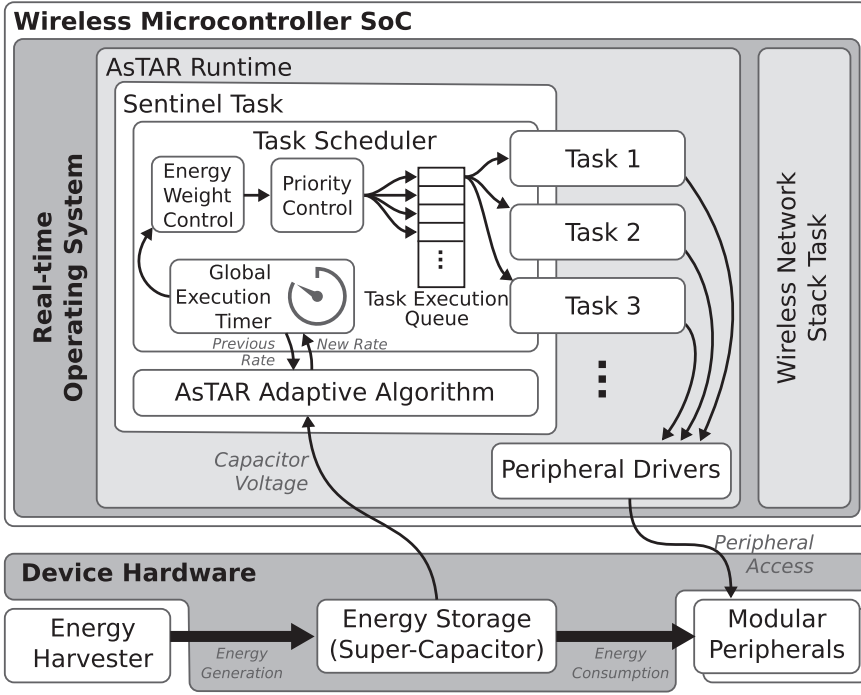


Fig. 5. AsTAR software architecture extends Real-time Operating Systems.

It should be noted that the execution of AsTAR tasks cannot be longer than the *Global Execution Timer*. AsTAR offers two underlying parameters that, in combination, ensure that task execution cannot be longer than the *Global Execution Timer*: (i) a parameterizable watchdog timer that pre-empts AsTAR tasks when a time limit (WT) is reached and (ii) a parameter (MT) that limits the maximum number of tasks that can be scheduled by AsTAR. The minimum period of the *Global Execution Timer* cycle (and therefore the highest task rate) is set to be larger than $WT \times MT$ to ensure that all tasks have sufficient time to execute with the active period of the device's duty cycle. It is the responsibility of the developer to ensure that their task completes its execution before the watchdog expires.

Algorithm 2 shows the pseudo code of the task scheduler. Each AsTAR task may be in one of three states: *task_ready*, *task_running*, and *task_blocked*. The *task_ready* state indicates that the task is ready to be executed by the AsTAR scheduler, *task_running* indicates that the task is running, and *Task_blocked* indicates that a task is waiting for an external (non-AsTAR) event and is not ready to execute. Every time the *Global Execution Timer* expires, AsTAR decrements the *Task Cycle Counter*. When the counter value reaches zero, if the task is in the *ready* state, it will be placed into the execution queue. If the task is in the *blocked* state, then it will be considered in subsequent execution rounds. A detailed evaluation of the behavior of this system can be found in Section 5.2.

3.4 AsTAR Software Stack

To achieve per-task energy accounting, AsTAR extends existing IoT OS with support for energy awareness. We have implemented prototypes of AsTAR on Real-time OS (FreeRTOS and uC/OS-II) and a preemptive multi-tasking OS (RIoT). AsTAR requires *preemption* to support the energy

watchdog described previously and therefore is unsuitable for cooperative schedulers, such as that used in the Contiki OS [6]. As shown in Figure 5, the core of AsTAR is the *Sentinel* task, which has two major building blocks: (i) the Adaptive algorithm that determines global task execution rates, and (ii) the Task Scheduler, who controls and manages the execution of all AsTAR tasks based upon their *Priority* and *Energy Weight* factors.

Each AsTAR task registers with the Task Scheduler, passing its developer-specified *priority*, *maximum* and *minimum* execution rate. The sentinel will then schedule each application task according to the algorithm described in Section 3.2, based upon its priority and maximum/minimum range. The API to register tasks is as follows:

register(task, priority, min_rate, max_rate),

where *task* is a reference to the task to be scheduled, *priority* assigns the developer-defined priority number to this task, *min_rate* specifies the minimum rate at which the task must run, and *max_rate* specifies the maximum rate at which the task may run. AsTAR Tasks are scheduled following the *Global Execution Timer*, which is updated every cycle by the AsTAR adaptive algorithm. All task rates are currently specified in executions per hour. AsTAR tasks may call any underlying system functionality, such as peripheral drivers and communication over the network. The sentinel considers all tasks to be black boxes and all energy accounting and control occurs at the level of individual task execution.

In cases such as data fusion where the output of multiple tasks must be combined, two solutions are possible. In the case of synchronous data fusion, multiple data gathering tasks may simply be modelled as sub-tasks of a single AsTAR task, which will then be scheduled in the standard way. In the case where these tasks must execute asynchronously, the tasks may communicate with each other via global data structure such as a message queue. In cases where tasks must run on a real-time schedule or execute only once, they should not be scheduled with AsTAR, but as a ‘background’ task using the underlying OS scheduler. This background energy consumption is neither measured nor controlled, however AsTAR tasks will adapt to this changing load by modifying their own behavior. The greater the proportion of functionality that is realized as AsTAR tasks, the finer the granularity with which energy can be managed.

3.5 AsTAR Hardware Platform

We have designed a minimalist battery-free Internet-of-Things hardware platform equipped with support for energy harvesting to evaluate AsTAR. This platform uses a super-capacitor for energy storage medium and may use various forms of harvested energy as its power supply. The platform can be configured to work with different application peripherals. The AsTAR software stack adapts to different sizes of a capacitor, a variety of peripherals, and harvesting sources.

The design of the AsTAR hardware platform strives for simplicity, it consists of only the basic elements of an energy-harvesting IoT platform, ensuring that it is easy to port the AsTAR approach to other platforms. A key feature of the AsTAR hardware platform is its modular design, which allows the energy harvester, super-capacitor, and application peripherals to be replaced. In the case of peripherals, these may also be replaced at runtime. Figure 6 provides an abstract level overview of the AsTAR hardware circuit, the major elements of which are explained below.

Energy Harvester: The AsTAR hardware platform adapts to various energy-harvesting mechanisms through a modular connector. AsTAR accepts voltages in the native working range of the system (e.g., harvesting from solar panels). Energy harvesters that generate AC voltages (e.g., vibration energy harvesting) or DC voltages outside of the working voltage range of system (e.g., **Thermoelectric Generators (TEGs)**) should be responsible for rectifying and converting the voltage to the working range. The AsTAR system makes no assumptions about the specifications

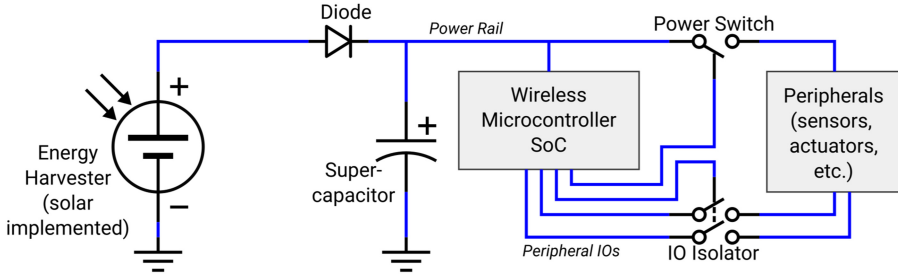


Fig. 6. Architecture of the minimalist AsTAR hardware platform. AsTAR can operate with various energy harvesters, super-capacitors, and peripherals.

of its energy harvester, which can be drop-in replaced without any software reconfiguration or modeling.

Super-capacitor Energy Storage: As can be seen in Figure 6, the AsTAR hardware platform uses a super-capacitor as its primary energy storage. The super-capacitor is charged by the energy harvester through a diode, to prevent reverse current flow when the energy harvester is at a lower voltage than the capacitor. We chose a schottky diode for their lower forward voltage drop. The AsTAR hardware platform design makes no assumptions about the specifications of its super-capacitor. The adaptive software stack introduced in Section 3.2 will adapt system behavior to suit the characteristics of this super-capacitor, such as the size of capacitor and leakage current.

Wireless Microcontroller: In principle, the AsTAR approach is compatible with any wireless microcontroller unit that offers sufficient memory to execute the system software, and an analog channel to monitor capacitor voltage.

Voltage Monitoring: Voltage measurement is critical to the system, since it is required by the AsTAR scheduler as discussed in Section 3.2. Naturally, voltage measurement with a higher resolution provides a finer grained adaptation control. The microcontroller used in our prototype has an internal voltage divider circuit connected to an ADC channel to measure the VCC of the chip, which in the design is connected directly to the super-capacitor. Our lab experiments show that the built-in 10-bit ADC is sufficient for this purpose.

Peripherals: The AsTAR platform supports modular application peripherals through a generic peripheral connector. Peripherals may include sensors, actuators, or even secondary networks. Modular peripheral connection provides flexibility to the platform so that the system can fit into different applications by adapting different peripherals. AsTAR system has no restrictions on peripheral energy demands, and the system adapts to changing energy demands and even changing types of connected peripherals.

4 IMPLEMENTATION

Section 4.1 describes key hardware implementation details for our prototype. Section 4.2 then describes implementation details of the AsTAR software stack.

4.1 Hardware Implementation

Figure 7 shows a finished prototype of the AsTAR device equipped with a solar panel. We selected the LTC5800-IPM from ADI to implement our prototype AsTAR platform. The LTC5800-IPM is an IETF Class-1 device that is realized as a highly integrated System on Chip (SoC), it includes the lowest power commercially available IEEE 802.15.4e radio unit and a 7.37 MHz ARM Cortex-M3 32-bit microprocessor with 12 kB RAM and 32 kB flash memory available for applications.

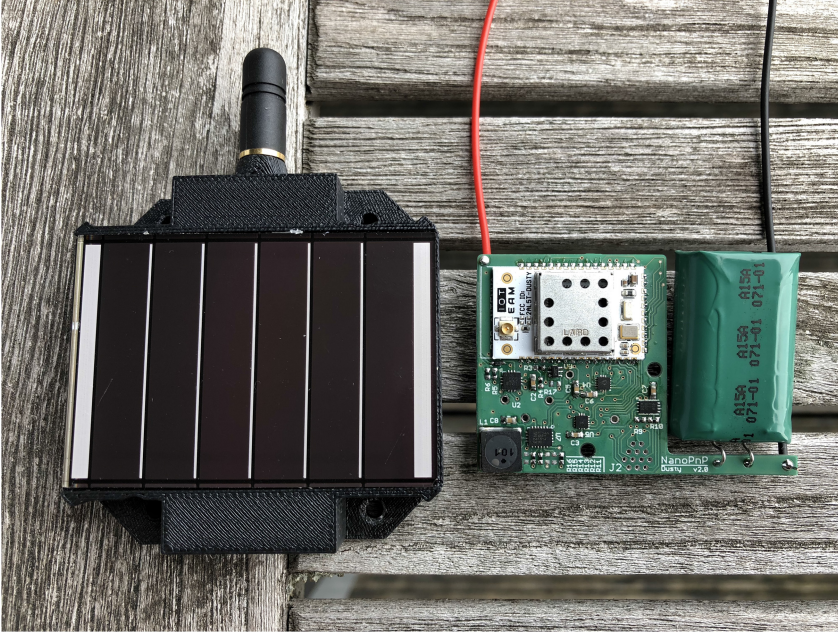


Fig. 7. Finished AsTAR device prototype with solar panel as energy harvester, together with the bare circuit board.

The LTC5800-IPM consumes 800 nA in deep-sleep mode, 1.3 mA in active mode, 4.5 mA when receiving a packet and 9.7 mA when transmitting a packet. The LTC5800-IPM runs the AsTAR software stack, as defined in Section 3.4.

Our experience has shown that super-capacitors with a low **Equivalent Series Resistance (ESR)** provide the best performance as (i) they can be charged easily from low current energy-harvesting sources and (ii) they can supply large currents without experiencing a significant voltage drop. This is necessary to support radios with high transmission power or high current sensors. Low leakage current is also essential for charge retention. The super-capacitors included in the prototype are the PHV-5R4 series from Eaton [7]. We chose them for their low ESR and high capacity. The 5F capacitor in our implementation leaks 10.2 μ A when operating at the defined OV at room temperature.

We evaluated our prototype implementation of AsTAR with controlled power supplies and solar energy harvesting. Other energy-harvesting techniques such as *thermal*, *RF*, and *kinetic energy harvesting* can be integrated with minimal hardware changes and will be the subject of our future work. In the case of solar tests, we use AM-5608CAR solar panel from Panasonic-BSG, a polycrystalline panel that generates up to 36 mA at 3.3 V under ideal operating conditions.

4.2 Software Implementation

We selected uC-OS/II from Micrium¹ as the OS for our prototype due to its integration with the LTC-5800-IPM and SmartMesh-IP. uC-OS/II also runs the SmartMesh IP network stack [30], a commercial implementation of **Time-synchronized Channel Hopping (TSCH)** and IPv6.

¹<https://www.micrium.com/rtos/kernels/>.

The SmartMesh IP stack is composed of IEEE 802.15.4e at the physical layer, TSCH at the link layer, and a standards-based IPv6-ready IoT *upper stack* (6LoWPAN, UDP). SmartMesh IP motes form a redundant low-power wireless multi-hop mesh network. The manager is responsible for building and maintaining the TSCH schedule of the network. By continuously monitoring the network and adapting the TSCH schedule to topological changes or different communication requirements, a SmartMesh IP network runs autonomously without human intervention. Matthys et al. [19] highlight the performance of SmartMesh IP, and demonstrates that it is possible to achieve battery lifetimes in excess of 6 years using a 2,400 mAh battery, even with plug-and-play peripheral support. The Key goal of AsTAR is to extend this lifespan.

The AsTAR runtime is compact and efficient, requiring very few state variables, including capacitor voltage and global task execution rate. The full evaluation of AsTAR's overhead can be found in Section 5.3.

5 EMPIRICAL EVALUATION

In this section, we evaluate AsTAR's performance. Section 5.1 presents the evaluation results of AsTAR's adaptive algorithm, Section 5.2 presents the result of AsTAR's multi-task scheduler, and Section 5.3 presents the overhead of AsTAR implementation.

5.1 Single-task Evaluation with Heterogeneity and Dynamism

In this section, we evaluate AsTAR's adaptive algorithm against the requirements identified in Section 2.4. Demonstrating that without any prior benchmarking and modeling, AsTAR can achieve its sustainability goals in the face of (i) platform heterogeneity and (ii) runtime dynamism, while (iii) incurring minimal memory and computation overhead. For evaluations in this section, AsTAR was configured with the following parameters: $SV = 2.7$ V, $OV = 3.7$ V. OV was selected to maximize sustainability and the energy efficiency of our platform based upon the specifications of the processor.

In respect of the wide range of current demands from different peripherals, we have chosen three representative load current settings: 5, 20, and 100 mA. For example, the 5 mA setting could represent a typical PT100 temperature sensor circuit, a three-axis **Inertial Measurement Unit (IMU)**, or an optical heart rate sensor; while peripherals such as an ultrasonic distance sensor, an optical object detector, or a RFID reader working at 13.56 MHz might fall into the 20 mA range; and the 100 mA setting could not only represent power-hungry sensors such as a camera, or a GPS receiver, but also actuator such as a servo motor. All of the peripherals have a fixed executing time of 1,500 ms for simplicity. The minimum task execution rate was set to 1 execution per hour, while the maximum task execution rate was set to 1,200 executions per hour. The energy watchdog timer was set to 1,600 ms to ensure that all tasks execute within the adaptation period. In the case of laboratory tests, we modified the supply current from 0.5 to 2 mA. In the case of real-world validation, the available power supply was determined by the harvesting capability of the AM-5608CAR solar panel.

In Section 5.2, we then focus on the behavior of AsTAR's Task Scheduler running multiple tasks and configured to support fair, prioritized and energy-aware scheduling.

5.1.1 Comparison to Exponentially Weighted Moving Average (EWMA). We choose to compare AsTAR to the commonly used EWMA approach applied to the harvested power supply, as this is a common approach that has been widely applied in related work such as References [12, 17, 24, 25]. This approach measures the input current from the energy source every time the *Global Execution Timer* expires and predicts the incoming current for the next period using the standard EWMA

Table 1. Benchmark Result of Mote Running EWMA Prediction-based Approach

Event	Calculating prediction and scheduling	Activating peripheral load	Transmitting packet	Sleep
Current (mA)	1.010 (I_{cal})	20.659 (I_{load})	0.890 (I_{tx})	0.067 (I_{sleep})
Duration (s)	0.739 (t_{cal})	1.533 (t_{load})	0.912 (t_{tx})	—

equation below:

$$I_{predict}(t+1) = \alpha \times I_{measure} + (1 - \alpha) \times I_{predict}(t),$$

where $I_{predict}(t+1)$ is the next predicted current, $I_{predict}(t)$ is the prediction from the previous cycle, $I_{measure}$ is the measured current, and α is the configurable smoothing factor between 0 and 1. The task execution rate is calculated to consume the amount of predicted energy with known task load consumption and platform leakage. Note that in this approach the mote assumes that peripheral power consumption is fixed and known in advance. In practice, EWMA defines the smoothing factor α based on the *a priori* benchmarks of the environment. To ensure that our results are fair, we select three smoothing factors, namely, 0.3, 0.5, and 0.7 in the experiments

We benchmarked the power consumption of the mote during four execution phases, including (1) calculating the prediction and updating global timer, (2) activating a 20 mA peripheral load, (3) transmitting packet, and (4) low-power sleep. The measurement is performed 10 times and the average values are calculated as shown in Table 1.

The prediction-based approach schedules each task so that it consumes the same amount of charge during the task period as was predicted to be harvested by the EWMA algorithm, as shown in the following equation:

$$I_{predict} \times t_{period} = I_{cal} \times t_{cal} + I_{load} \times t_{load} + I_{tx} \times t_{tx} + I_{sleep} \times (t_{period} - t_{tx} - t_{load} - t_{tx}).$$

The t_{period} is the calculated task period, I_{cal} , I_{load} , I_{tx} , and I_{sleep} are the current during calculating, load, transmission, and sleep phases, respectively, while t_{cal} , t_{load} , and t_{tx} are the durations of each phase accordingly. The task period can therefore be calculated as follows:

$$t_{period} = \frac{I_{cal} \times t_{cal} + I_{load} \times t_{load} + I_{tx} \times t_{tx} - I_{sleep} \times (t_{tx} + t_{load} + t_{tx})}{I_{predict} - I_{sleep}}.$$

The task is then executed and the *Global Execution Timer* is updated with the new *Task_Period*. The task execution rate is computed as

$$R_{task} = \frac{1 \text{ hour}}{Task_Period}.$$

Note that the EWMA prediction-based approach can neither deal with dynamic task loads nor dynamic platform leakage currents. Thus, its calculation is considerably more complicated than that of AsTAR.

To create a common ground for the comparison, both AsTAR and the prediction-based approach are implemented on the same platform with the same capacitor size, leakage current, and network connection, with the exception that the prediction-based approach is equipped with a current sensor to measure the input current. The same energy input is applied to all approaches, depending upon the experiment. AsTAR measures voltage with 5.4 mV accuracy, and the prediction-based approach measures current with 100 μ A accuracy using an INA219 current sensor [27].

The approaches are evaluated using two criteria: *Efficiency* and *Sustainability*. *Efficiency* in energy-harvesting adaptation approaches is defined as the ability to convert energy into task executions. *Sustainability* is defined as the ability to maintain a user-defined charge level in the capacitor. Naturally a system that executes more tasks with the same energy input is considered

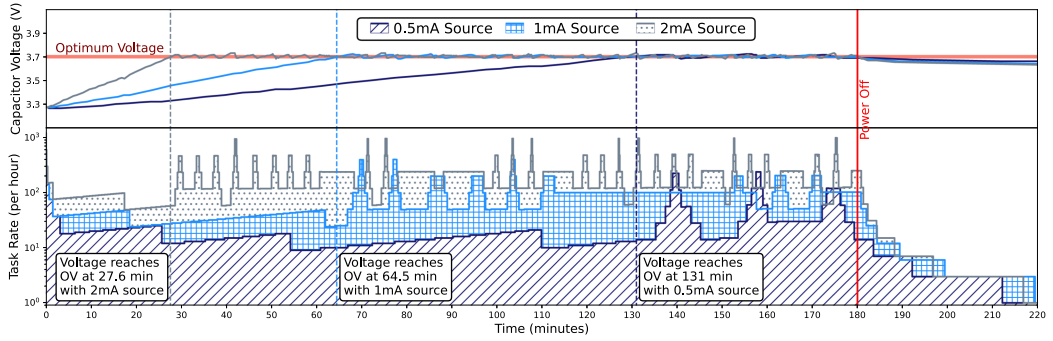


Fig. 8. Heterogeneous power supply: 0.5, 1, and 2 mA with fixed 5 F capacitor and 20 mA peripheral load.

more efficient, while a system is sustainable as long as the capacitor voltage level does not fall below the Optimum Voltage threshold. Both approaches are tested for the same amount of time using the same parameters, allowing for a side-by-side comparison.

5.1.2 Adapting to Platform Heterogeneity. We first evaluate AsTAR with respect to three key dimensions of platform heterogeneity: (i) *available energy input*, (ii) *peripheral power draw*, and (iii) *capacitor size*, and we also compare AsTAR with the prediction approach in the same setup. In all cases the goal of AsTAR is to ensure consistent and sustainable operation in the face of this heterogeneity and without the need for any platform specific modeling. Figures 8–10 follow a common timeline, which begins at 0 min with motes at 3.3 V, in the Low-voltage state. AsTAR charges the capacitor to the *OV* state, which it then aims to maintain until power is shut off at 180 min. At that point, AsTAR detects the lack of power and rapidly throttles task execution rates. Each graph shows the super-capacitor voltage, which is presented on a linear scale, and the task execution rate, which is presented on a log scale due to its large variance. It should be noted that our power supply hardware has an inherent inaccuracy of up to 5% in all configurations, which causes a low level of background “noise” in all of our experiments.

Heterogeneity in Energy Input: Figure 8 shows the impact that different power supplies have on the operation of AsTAR with a fixed 5 F super-capacitor and 20 mA peripheral load. With 0.5 mA supply, AsTAR takes 131 min to charge from 3.3V to *OV* with an average execution rate of 16.9 executions per hour, and it achieves on average 50 executions per hour after reaching *OV*. With 1 mA supply, AsTAR takes 64.5 min to charge to *OV* with an average execution rate of 39 executions per hour, and it achieves on average 100.6 executions per hour after reaching *OV*. With 2 mA supply, AsTAR takes 27.6 min to charge *OV* with an average execution rate of 80.4 executions per hour, and it achieves on average 203.3 executions per hour after reaching *OV*.

It can be seen that the average task execution rate after reaching *OV* roughly corresponds to the input power, with small differences caused by the sleep current of the mote. However, it should be noted that during the charging phase the mapping between the input power and the task rate is indirect as AsTAR only guarantees that charge accumulates in that state, rather than attempting to guarantee the rate at which charge accumulates. The classic saw-tooth pattern of **Additive Increase Multiplicative Decrease (AIMD)** [5] can be seen clearly in all three cases. For all power supply levels, it can be seen that AsTAR ensures a smooth accumulation of charge and is extremely stable in the *OV* state, demonstrating a maximum deviation of 30 mV or 0.81% from the optimum. When power is cut off at 180 min, AsTAR rapidly backs off to the minimum task execution rate of 1 task per hour, where it remains until power is either restored or voltage levels fall into the

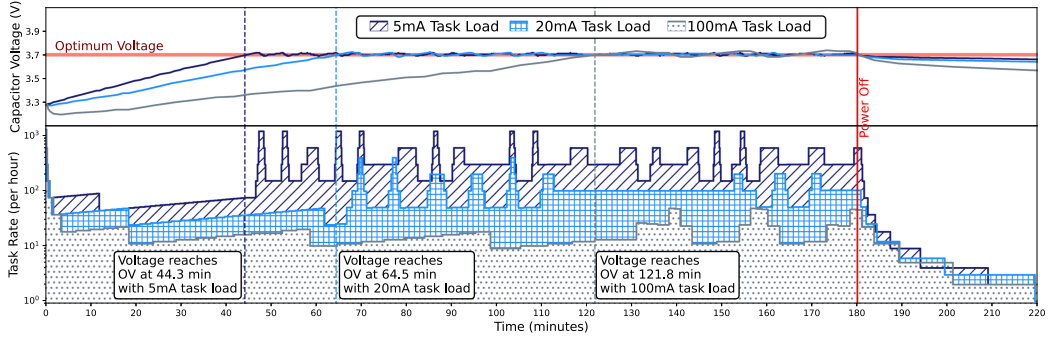


Fig. 9. Heterogeneous peripheral demand: 5, 20, and 100 mA with fixed 5 F capacitor and 1 mA power supply.

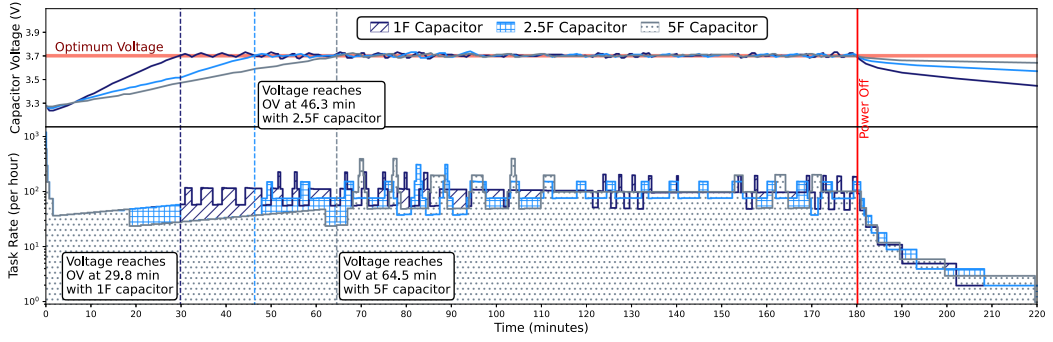


Fig. 10. Heterogeneous capacitor sizes: 1, 2.5, and 5 F with fixed 20 mA peripheral load and 1 mA power supply.

SV state at 2.7 V. AsTAR clearly achieves its optimization goals of acquiring and sustaining an optimal charge in the face of $4\times$ heterogeneity in power supply current.

Heterogeneity in Peripheral Power Demand: Figure 9 shows the impact that different peripheral power demands have on the operation of AsTAR with a fixed 5 F super-capacitor and a 1 mA power supply. With 5 mA peripheral load, AsTAR takes 44.3 min to charge from 3.3 V to OV with an average execution rate of 67.7 executions per hour, and it achieves on average 373.9 executions per hour after reaching OV. With 20 mA peripheral load, AsTAR takes 64.5 min to charge to OV with an average execution rate of 39 executions per hour, and it achieves on average 100.6 executions per hour after reaching OV. With 100 mA peripheral load, the voltage level temporarily goes down due to the high starting task rate, AsTAR quickly adapts the load by applying Multiplicative Decrease. In the end it takes 121.8 min to charge OV with an average execution rate of 16.2 executions per hour, and it achieves on average 20.7 executions per hour after reaching OV.

For all peripheral test loads, AsTAR ensures a smooth accumulation of charge and remains stable in the OV state, with a maximum deviation of 37 mV or 1%. When power is cut off, AsTAR backs off quickly to the minimum task execution rate. These results show that AsTAR remains robust in the face of heterogeneous peripheral loads, such as those arising from sensor, actuator and radio peripherals. In the case of platforms that support the plug-and-play modification of peripherals at runtime such as Flicker [11] and μ PnP [31], AsTAR will even adapt this behavior at runtime as described in Section 5.1.3.

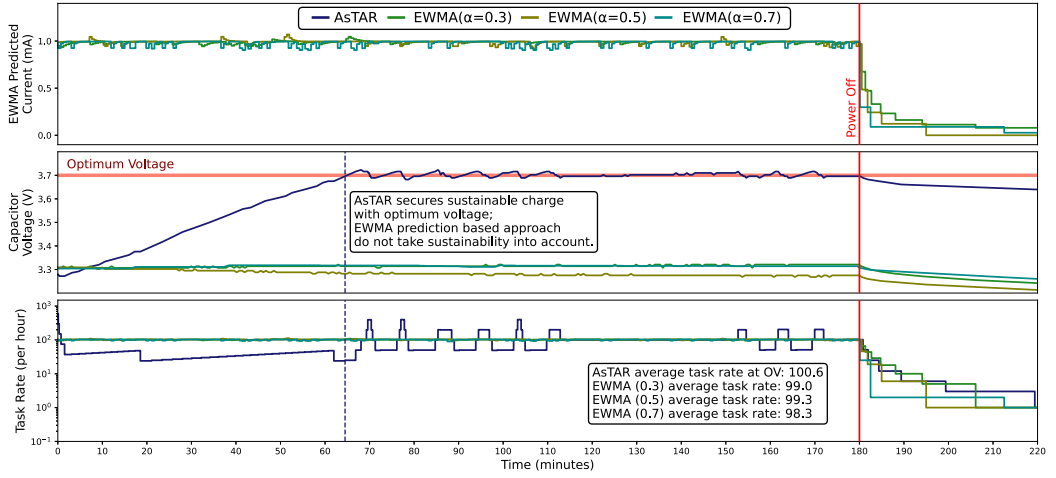


Fig. 11. EWMA prediction-based approach could not prioritize a optimum charge.

Heterogeneity in Capacitor Size: Figure 10 shows the impact that different capacitor sizes have on the operation of AsTAR with a fixed 20 mA peripheral load and a 1 mA power supply. With a 1 F capacitor, AsTAR takes 29.8 min to charge from 3.3 V to OV with an average execution rate of 54.3 executions per hour, and it achieves on average 96.7 executions per hour after reaching OV. With a 2.5 F capacitor, AsTAR takes 46.3 min to charge to OV with an average execution rate of 44 executions per hour, and it achieves on average 94.3 executions per hour after reaching OV. With a 5 F capacitor, AsTAR takes 64.5 min to charge OV with an average execution rate of 39 executions per hour, and it achieves on average 100.6 executions per hour after reaching OV.

For all capacitors, AsTAR ensures a smooth accumulation of charge, stable operation in the OV state (maximum deviation of 30 mV or 0.81%) and rapidly throttles task execution rates to their minimum rate after power being cut off. As can be seen from Figure 10, the major trade-off in capacitor sizes is how quickly they discharge when power is removed, which varies in direct proportion to their capacitance.

Comparison to EWMA prediction: Figure 11 shows the result of AsTAR alongside the EWMA approach with 3 smoothing factors (0.3, 0.5 and 0.7), using a 5 F capacitor, a fixed 20 mA peripheral load and a 1 mA power supply. As we have already seen, AsTAR charges the capacitor to OV at 64.5 min after start, and it achieves on average 100.6 executions per hour after reaching OV. In the case of EWMA, the input current is predicted correctly, achieving task rate of 99, 99.3, and 98.3 accordingly. However, since all available energy is converted into task execution, these approaches do not charge the capacitor to OV, failing the *Sustainability* criterion, making the devices less robust against power failure.

As can be seen from Figure 11, AsTAR always prioritizes the acquisition of a sustainable amount of charge, while prediction-based approaches do not. Under a fixed power supply, prediction-based approaches are naturally quite accurate. It can be seen that once AsTAR reaches OV, the average task execution rate is very close to that of prediction-based approaches.

Discussion: As can be seen from the evaluation presented above, AsTAR is capable of ensuring sustainable energy-harvesting operation even in the face of heterogeneity of power supplies, peripherals and energy storage capacity. AsTAR ensures that devices rapidly accumulate and then maintain an optimum charge level, adjusting quickly when power is lost. Considering the experiments performed above, it can be seen that AsTAR allows developers considerable freedom in

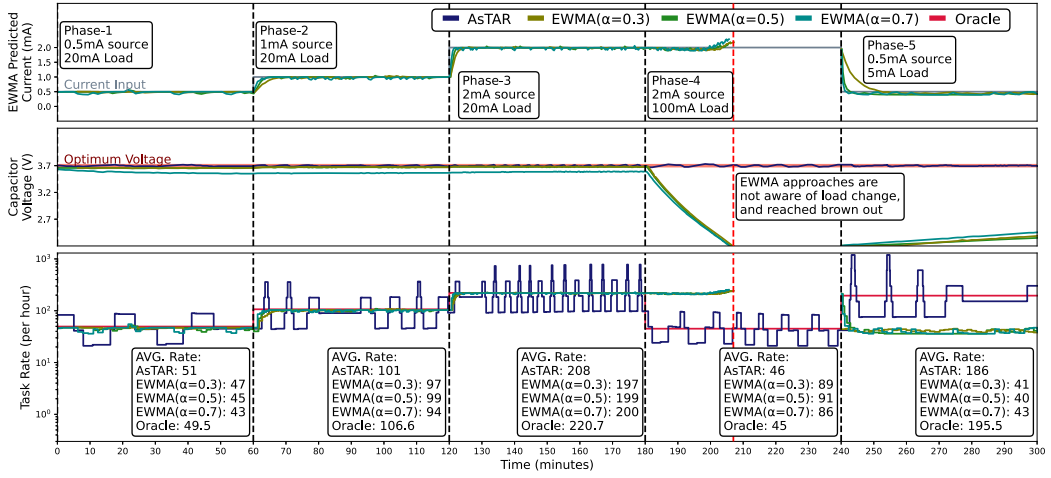


Fig. 12. Robustness of AsSTAR equipped with 5 F capacitor to runtime dynamism in power supply and demand, in comparison with the Oracle system.

their energy-harvesting designs, while providing consistent behavior. This enables developers to, for example, select larger capacitors for applications with longer typical power outages or smaller capacitors where rapid re-charging after power outages was considered more important. Considering peripherals, developers are relieved from peripheral-specific power consumption concerns. This is critical for plug-and-play platforms [11, 31], for which new peripherals may be developed even after platforms have been deployed in the field. In contrast to EWMA energy-harvesting predictions, which require a known peripheral load, AsSTAR achieves its goals without the need for any platform-specific benchmarking or modeling before runtime, which we believe eliminates a significant barrier to entry for the building sustainable energy-harvesting systems. In Section 5.1.3, we next evaluate the robustness of AsSTAR in the face of runtime dynamism.

5.1.3 Adapting to Runtime Dynamism. To evaluate the effect of runtime dynamism on the behavior of AsSTAR, we created two scenarios: (i) *Dynamism across fixed-time stages* and (ii) *Dynamism that varies continually with time*. Figures 12 and 13 present the result for these scenarios.

To provide context on the performance of both approaches, we also introduced a theoretical ideal scheduler: the *Oracle*, which knows exactly when and how much energy is going to be available, as well as the amount of energy one task operation consumes, enabling it to make perfect scheduling decisions about task operating rates. The Oracle is also free to schedule tasks with perfect precision, allowing for arbitrary sample periods. In this way the Oracle maintains an optimum charge level at all times when there is sufficient energy income to do so.

Dynamism with fixed time stages: In this evaluation, we applied five scenarios with different energy supply and demand characteristics in series for all approaches. Each phase takes one hour and the setup is constant during each phase. Figure 12 shows the results of this evaluation:

- **Phase-1:** The experiment begins in the *OV* state with a 20 mA peripheral load and a 0.5 mA power supply. AsSTAR schedules the task execution rate so that the voltage stays in the *OV*. All prediction-based approaches perform equally well in this phase, following the current input accurately, resulting in almost no voltage change. Both approaches achieve results that are close to the Oracle (within 3% for AsSTAR, worst case 13 % for EWMA).

- **Phase-2:** At 60 min, the supply current is doubled to 1 mA. AsTAR quickly adapts the task execution rate, and on average it scheduled 101 executions. EWMA with $\alpha = 0.3$ adapts slower than $\alpha = 0.5$ and $\alpha = 0.7$ due to the different weight that they place on new data. Both approaches achieve results that are very close to the Oracle, though EWMA performs slightly worse than AsTAR as it is slower to adapt to change.
- **Phase-3:** At 120 min, the supply is doubled again to 2 mA. The result is similar to *Phase-2*. Note that the adaptation is quicker for EWMA than last change as the execution is more frequent.
- **Phase-4:** At 180 min, the supply stays unchanged, but the peripheral load changes from 20 to 100 mA. AsTAR quickly adapts to the new load, producing results that are very close to the Oracle. However, the EWMA approaches cannot consider peripheral dynamism in runtime. Hence they follow the same task rate as in the last phase and naturally the motes brown out due to the higher consumption. Therefore, EWMA approaches fail the *Sustainability* goal. The EWMA motes are kept in brown-out state for the remainder of this phase.
- **Phase-5:** At 240 min, we changed both the supply and demand, namely, 0.5 mA power supply and 5 mA peripheral load. AsTAR adapts to the change quickly, resulting in 186 executions per hour on average, which is only 5% less than the Oracle. EWMA motes adapt to the change in input current, without knowledge of the 4× smaller peripheral load, hence achieving far less executions (best case 78 % less than the Oracle) and thus failing the *Efficiency* goal.

In conclusion, both AsTAR and EWMA prediction-based approaches are able to schedule close to perfectly under a fixed peripheral load and constant supply. AsTAR also handles peripheral load dynamism well, but the EWMA-based approaches cannot. We further note that EWMA approaches achieve these result with extra hardware support for current measurement, as well as detailed benchmarks, which AsTAR does not need.

Continuously varying dynamism: In this part of evaluation, we introduce further dynamism, by applying a supply input that is random on both amplitude and duration, while keep the load constant. The input current is randomly selected between 0 and 5 mA, and the duration of each step is randomly selected between 1 and 5 s, with each experiment lasting 20 min. The input current is generated using a Keysight B2901A programmable power supply [15], and the same random input current is applied on all approaches. The result can be seen in Figure 13.

Figure 13 shows four sub-figures, namely, the actual input current, the EWMA predicted current, the capacitor voltage, and the scheduled execution rate. As indicated in the figure, the EWMA approaches failed to follow the input current waveform, because of its random and fast changing nature. As a result, they all perform sub-optimally in comparison to AsTAR and the Oracle, with up to 57.7% less executions per hour. AsTAR, however, does not rely on instantaneous current data and therefore achieves 246 executions per hour on average, which is only 3% less than the Oracle. Note that, since the EWMA approach cannot follow the exact pattern of the input current, their performance is non-deterministic and the results may be better or worse depending on the timing of each experiment and the exact waveform. We believe that this makes them impractical for real-world deployment in highly dynamic environments.

Discussion: Considered in sum, AsTAR reacts rapidly in both continuous and staged dynamism scenarios, ensuring that charge levels remain optimum for the vast majority of the experiment timeline. Throughout the experiment, AsTAR is remarkably robust to changes in power supply and demand, even in the case of very large or random runtime reconfigurations that impact both the supply and demand side of the energy equation. Compared to the Oracle, which offers ideal performance, AsTAR may over-shoot or under-shoot at the beginning of each phase, but it quickly

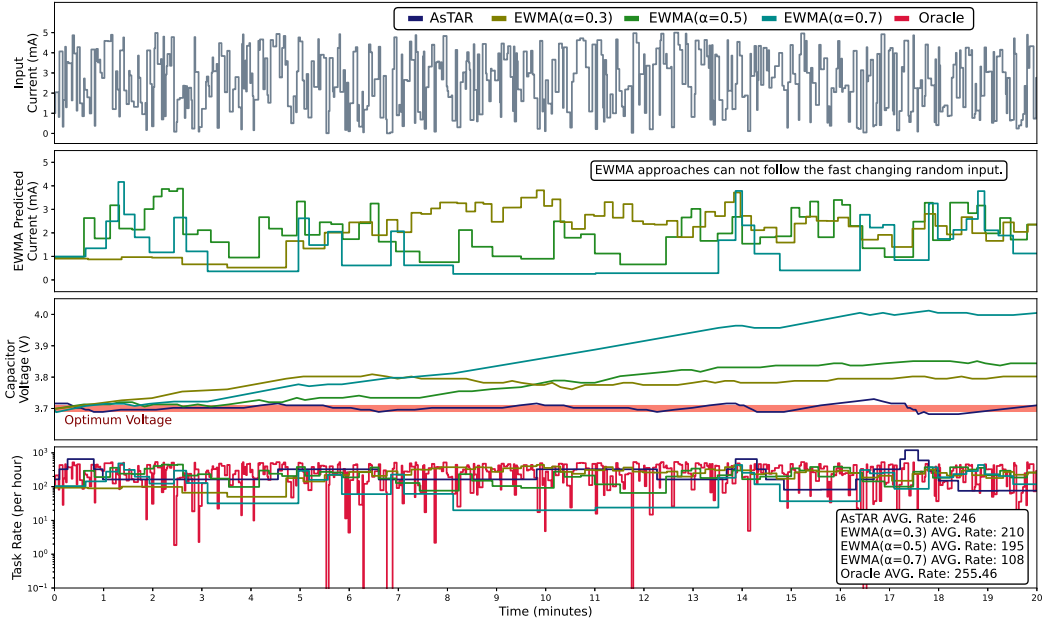


Fig. 13. Robustness of AsSTAR equipped with 5 F capacitor to runtime dynamism in power supply and demand, in comparison with the Oracle system.

reacts to the situation and restores the optimum charge, and remains close to the ideal task execution rate on average.

In general, EWMA-based approaches work well with slow moving or fixed energy patterns, with the cost of adding a current sensor in hardware, more complicated calculations, and a carefully selected smoothing factor α . Critically, while it is possible to find configurations of EWMA prediction that work well in a given scenario, EWMA prediction approaches cannot adapt to fast changing input current, and requires the task load and leakage current to be *known constants*, thereby forbidding run-time reconfigurations on the platform.

Section 5.1.4 applies AsSTAR in a real-world solar energy-harvesting case study to validate that these properties hold outside of the lab.

5.1.4 Real World Validation. This section evaluates AsSTAR in a real-world energy-harvesting scenario. We deployed a solar powered AsSTAR device running the software stack indoors, next to a window of our office building for a period of two days. This device is equipped with a 5 F capacitor and a sensing peripheral that consumes 100 mA for 100 ms when sampling. In addition to monitoring capacitor voltage and task scheduling rates, we also logged light levels on a separate device using a standard light sensor that is deployed at the same location. It should be noted that this light sensor provides only an indication of available solar energy rather than a one-to-one mapping of the energy produced by the solar panel. As our office is surrounded by other buildings, direct sunlight is available for only a small part of the day, with reflected light for the remainder of the time. During the two days of the experiment the weather was predominantly cloudy with intermittent sunny periods.

Figure 14 shows the results of this experiment. Capacitor voltage is plotted at the top, followed by task scheduling rates in the middle and light levels at the bottom. Capacitor voltage is plotted on a linear scale. Task rates and light levels were plotted on a log scale due to their large variance.

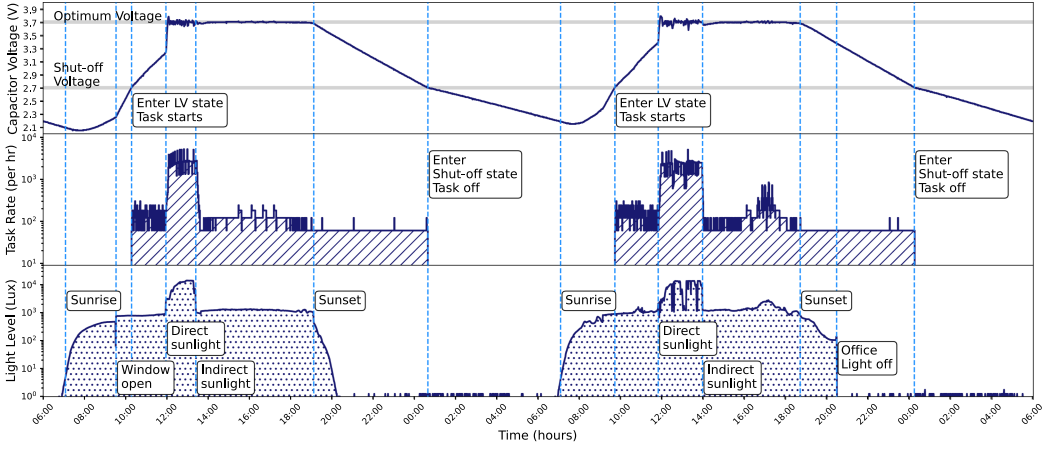


Fig. 14. A section of 2 full days from the real-world deployment. The AsTAR device is equipped with a 5 F capacitor, a peripheral that demands 100 mA current, and it uses only solar energy harvested from an indoor location. The three curves each show the capacitor voltage, the application task rate, and light level recorded by the light sensor.

This figure again shows that AsTAR achieves its goals of acquiring and sustaining an optimal charge whenever sufficient energy is available, and even in the face of large fluctuations in available energy. We now highlight interesting elements from our two-day experiment.

- **The sun rises** at around 7 a.m. on both days. After a night of darkness, the mote is in the SV mode to preserve operation and no application tasks are scheduled. As the sun rises, light levels quickly increase to around 1,000 lux and the capacitor charges past the SV threshold.
- **Indirect sunlight:** Low-voltage state mode begins at around 10 a.m. on both days as the mote voltage rises past the shut-off threshold of 2.7 V. The AsTAR scheduler applies AIMD to reach an optimal task rate of approximately 150 executions per hour while the capacitor charges in somewhat consistent indirect sunlight.
- **Direct sunlight** falls upon the panel at around noon for a period of between 1 and 2 h, depending on weather conditions. This is clearly visible in the light level graph, which increases by more than an order of magnitude from under 1,000 lux to over 15,000 lux. This causes the mote to rapidly accumulate charge and reach the optimum level of 3.7 V. AsTAR handles this by rapidly increasing task rates to 2460 executions per hour, which sustains the voltage level at the optimum value of 3.7 V, with a variance during this period of 70 mV or 1.9%. The larger variance during this period arises due to clouds unpredictably obscuring the sun.
- **Falling light levels:** When the period of direct sunlight ends in the early afternoon, light levels again fall by more than an order of magnitude from around 15,000 lux to 1,000 lux, which briefly pulls the voltage below OV, causing AsTAR to quickly adapt the task rate to an average of around 150 executions per hour. This adaptation is sufficient to ensure a deviation of less than 40 mV or 1.1% from the optimum.
- **Increased reflected light** occurs on day two due to high white clouds from late in the afternoon. Light levels more than double from 1,000 lux to 2,400 lux. This causes the voltage to briefly enter the HV zone and AsTAR applies MIAD to increase task rates to 276 executions per hour. This compensates for the increased light levels, resulting in a deviation of less than 20 mV or 0.55% from the optimum.

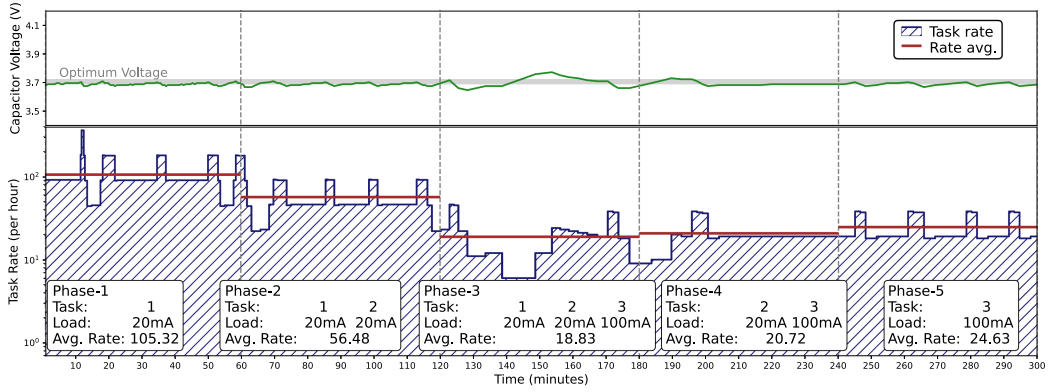


Fig. 15. Evaluation result of multi-task scheduling with Round-robin, using tasks that have different loads.

- **The sun sets** around 7 p.m. on both days, causing the light level to quickly drop towards 0 lux, which prevents the accumulation of charge. AsTAR reacts by rapidly cutting the task rate using *Multiplicative Decrease* to the developer-specified minimum of 60 task executions per hour.
- **Night time power saving** begins between midnight and 1AM as the capacitor voltage falls below 2.7 V, causing AsTAR to enter the SV mode until the sun rises again and charges the capacitor past the SV threshold. As can be seen from the figure, disabling the task significantly slows the discharge rate of the capacitor, enabling the device to survive 12 h of darkness.

In all phases of the experiment, AsTAR behaved as expected. During this evaluation, it remained remarkably robust against the radical changes in available energy levels that arise due to specifics of the deployment location. This is accomplished without the need for any prior benchmarking and modeling. Furthermore, while solar modeling approaches [2, 26] provide an accurate prediction of solar energy at a given location and time-of-year, they cannot take into account deployment specifics such as the impact of buildings and local weather.

5.2 Multi-tasking Evaluation

We evaluated AsTAR's scheduler in a variety of multi-tasking scenarios with Round-robin, prioritized, and energy-aware scheduling. All scenarios in this section are conducted in a realistic lab environment.

For the evaluations in this section, AsTAR was configured with the following parameters: $SV = 2.7$ V, $OV = 3.7$ V. The test peripherals are created with a representative range of current demands (5, 20, and 100 mA) and a fixed sensing cycle of 1,500 ms. The energy watchdog timer was set to 1,600 ms for each task in the system to ensure that all tasks execute within the adaptation period. The power supply is set to fixed 1 mA with the same inherent inaccuracy of 5% as in our previous experiment.

5.2.1 Round-robin Scheduling. In the case of round-robin scheduling, all tasks are assigned an equal rate. The test comprises five phases, as shown in Figure 15.

- **Phase-1:** The experiment begins in the OV state with only one task that has a 20 mA load. AsTAR configures task execution rate at an average of 105.32 executions per hour, and the maximum deviation of capacitor voltage is 31 mV or 0.81%.

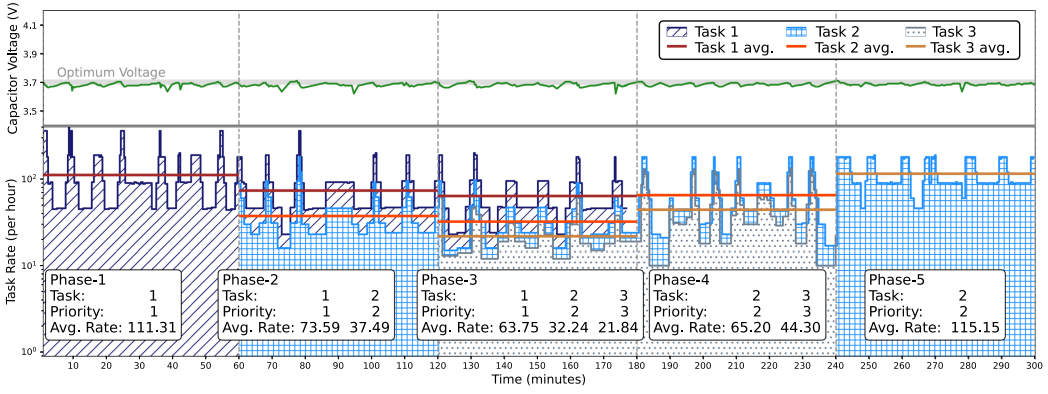


Fig. 16. Evaluation of multi-task scheduling with priorities, using tasks that have the same load.

- **Phase-2:** At 60 min, the second task with an equal 20 mA load is registered in the system. AsTAR quickly responds by scheduling both tasks at a slower rate of 56.48 executions per hour. The maximum deviation of the capacitor voltage from the optimum is 38 mV or 1.03%. Both tasks receive the exact same execution rate, which is roughly half of the rate in *Phase-1*.
- **Phase-3:** At 120 min, we register another task with an associated 100 mA load with AsTAR. Due to the larger load of this task, it generates a greater deviation of 66 mV or 1.78% from the optimum voltage, however, AsTAR quickly recovers. In this phase, tasks are scheduled on average 18.83 per hour.
- **Phase-4:** At 180 min, Task 1 is removed from the system, leaving Task 2 and 3. AsTAR increases the scheduling rate to an average of 20.72, effectively sharing the energy surplus arising from the removal of Task 1 with Task 2 and 3. The maximum deviation of capacitor voltage is 24 mV or 0.65%.
- **Phase-5:** At 240 min, Task 2 is removed from the system, leaving only Task 3. AsTAR allocates the resulting energy surplus to the remaining task, increasing the average execution rate to 24.63. The maximum deviation of capacitor voltage from the optimum is 66 mV or 1.78%.

As shown in Figure 15, throughout the evaluation, AsTAR ensures that the energy reserve of the system remains close to the optimum voltage, while scheduling heterogeneous tasks equally. For example in *Phase-2*, scheduling two equal tasks resulted in an execution rate that was half as frequent for both tasks. In *Phase-3*, when a heavier task is introduced into the system, the execution of every task is reduced to a lower but equal rate.

5.2.2 Prioritized Scheduling. As we discussed in Section 3.3 and demonstrated in Section 5.2.1, it may not be desirable for all AsTAR tasks to execute at the same rate. AsTAR also enables developers to prioritize task execution. The following experiment evaluates the performance of prioritized task scheduling for a set of equal tasks, with a load of 20 mA. The results are shown in Figure 16 and the five phases of the experiment are described below.

- **Phase-1:** The experiment begins in the OV state with one task with priority 1. AsTAR configures task execution rate at an average of 111.31 executions per hour and the maximum deviation of capacitor voltage is 66 mV or 1.78%.
- **Phase-2:** At 60 min, a second task with a priority of 2 is registered in the system. In AsTAR a task with priority 2 executes half as frequently as a task with priority 1. AsTAR responds to

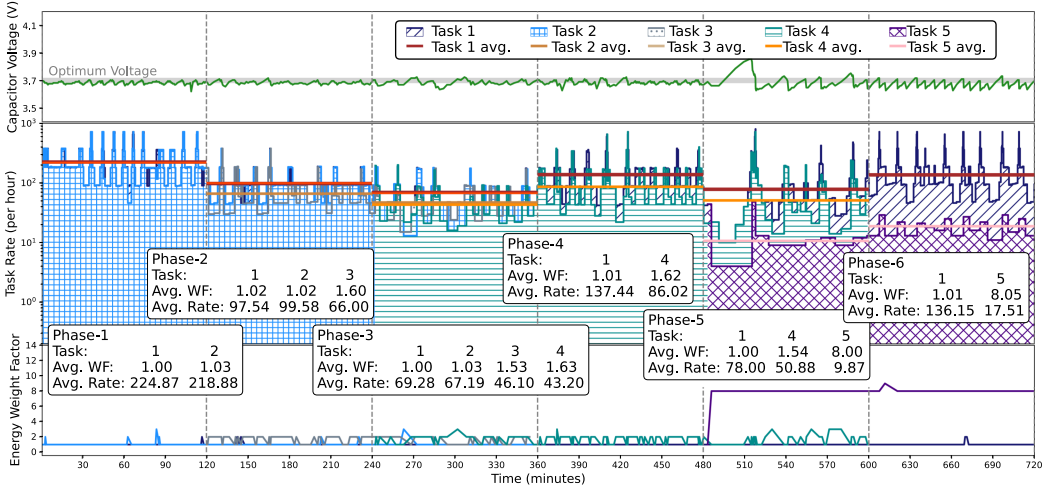


Fig. 17. Evaluation of multiple task scheduling with different loads and energy weight factors.

the new task by scheduling both tasks slower, on average Task 1 is rescheduled at 73.59 per hour, and task 2 is scheduled at 37.49 per hour. The maximum deviation from the optimum voltage is 80 mV or 2.16%. The ratio of task execution schedules is within 1.85% of the ideal.

- **Phase-3:** At 120 min, we register a second task with priority 3 into the system. AsTAR schedules these three tasks following their priorities with Task 1 at 63.75, Task 2 at 32.24, and Task 3 at 21.84 on average. The maximum deviation of capacitor voltage is again 80 mV or 2.16%. Task 1 runs at double the rate of Task 2 and triple the rate of Task 3, reflecting their assigned priority numbers. The ratio of task execution schedules is within 2.7% of the ideal.
- **Phase-4:** At 180 min, Task 1 is removed from the system, leaving Tasks 2 and 3. AsTAR reschedules Task 2 to 65.2 execution per hour, and Task 3 to 44.30 per hour on average. The maximum deviation of capacitor voltage from optimum is 31 mV or 0.84% and the ratio of task execution schedules is within 1.88% of the ideal.
- **Phase-5:** At 240 min, Task 2 is removed from the system. AsTAR adjusts the schedule of Task 3 to 115.15 executions per hour, which is roughly the same as *Phase-1* when only Task 1 was running. The small deviation is likely due to variances in the lab power supply and changes in the background energy consumption of the network stack. The maximum deviation of capacitor voltage from optimum is 66 mV or 1.78%.

As shown in Figure 16, throughout the evaluation, AsTAR honors the developer-specified task priorities, while ensuring that the energy reserve of the system remains close to the optimum voltage at all times.

5.2.3 Energy-weighted Scheduling. In this section, we evaluate the AsTAR's support for energy-weighted scheduling using a total of five tasks. Tasks 1 and 2 have the same load of 5 mA, Tasks 3 and 4 have a load of 20 mA, and Task 5 has a load of 100 mA. The results are shown in Figure 17 and the six phases of operation are described below.

- **Phase-1:** The experiment begins in the OV state with Tasks 1 and 2 registered, they both have the same load of 5 mA. AsTAR identifies their energy weight factors to be 1.00 and 1.03 on average, and schedules Task 1 at 224.87 executions per hour and Task 2 at 218.88 per hour on average. The maximum deviation of capacitor voltage from the optimum is 80 mV

or 2.16%. As Tasks 1 and 2 have a low and equal energy consumption, they are allocated the same schedule. The ratio of task execution schedules is within 0.26% of the ideal.

- **Phase-2:** At 120 min, Task 3 is registered with a load of 20 mA. AsTAR recalculates the energy weight factor of Tasks 1 to 3 as 1.02, 1.02, and 1.6, resulting in execution rates of 97.54, 99.58, and 66.00 executions per hour, respectively. The maximum deviation of capacitor voltage is 45 mV or 1.22% from the optimum. The ratio of task execution schedules is within 5.79% of the ideal.
- **Phase-3:** At 240 min, Task 4 is registered with the system using a load of 20 mA. AsTAR recalculates the energy weight factors of Task 1 to Task 4 as 1.00, 1.03, 1.53, and 1.63, respectively, and then schedules them at an average hourly execution rate of 69.28, 67.19, 46.10, and 43.20, respectively. The maximum deviation of capacitor voltage in this period is 66 mV or 1.78% from the optimum. The ratio of task execution schedules is within 1.88% of the ideal.
- **Phase-4:** At 360 min, Tasks 2 and 3 are removed from the system. AsTAR recalculates the energy weight factors of Tasks 1 and 4 to be 1.01 and 1.62 on average, and then schedules them at an average execution rate of 137.44 and 86.02 per hour, respectively. The maximum deviation of capacitor voltage is 73 mV or 1.97% from the optimum. The ratio of task execution schedules is within 0.39% of the ideal.
- **Phase-5:** At 480 min, Task 5 is registered with the system using a heavy load of 100 mA. AsTAR identifies energy weight factors of Tasks 1, 4 and 5 to be 1.00, 1.54, and 8.00 on average, and then schedules them at 78.00, 50.88, and 9.87 per hour on average. Because of the heavy load, AsTAR significantly reduces the execution rate, which causes a temporary overshoot of the optimum voltage, the maximum deviation of capacitor voltage in this period is 163 mV or 4.41%. The ratio of task execution schedules is within 1.22% of the ideal.
- **Phase-6:** At 600 min, Task 4 is also removed from the system, leaving Tasks 1 and 5. AsTAR recalculates the energy weight factors of Tasks 1 and 5 to be 1.01 and 8.05 on average, and then schedules them at 136.15 and 17.51 executions per hour on average, presenting roughly the same ratio. The maximum deviation of capacitor voltage is 73 mV or 1.97% in this period. The ratio of task execution schedules is within 2.44% of the ideal.

As with fair and prioritized scheduling, AsTAR ensures that the energy reserve of the system remains close to the optimum at all times. However, in contrast to the previous experiments, the impact of energy-hungry tasks on the other tasks is minimized. For example, when Task 5 is introduced in *Phase-5*, the impact on the other tasks is less significant compared to the behavior in Section 5.2.1. This effectively addresses the tragedy of the coulombs. In the following section, we evaluate the combination of prioritized and energy-aware scheduling.

5.2.4 Prioritized and Energy-weighted Scheduling. In this section, we combine the energy weight and prioritization systems. The experiment consists of five tasks. Tasks 1 and 2 have the same load of 5 mA, Tasks 3 and 4 have a load of 20 mA, and Task 5 has a load of 100 mA. Tasks 1–5 are assigned priorities of 1, 2, 1, 2, and 2, respectively. Figure 18 shows the results of the experiment, the six phases of which are discussed below.

- **Phase-1:** The experiment begins in the OV state with Tasks 1 and 2 registered. Both tasks have a load of 5 mA. Task 1 is assigned priority 1, and Task 2 is assigned priority 2. AsTAR identifies their energy weight factors to be 1.01 and 1.00 on average. As the energy weights of the task are so close, their execution rates are largely determined by their priorities. AsTAR schedules Task 1 at 292.04 executions per hour and Task 2 at 149.64 per hour on average. The allocation of these two task schedules closely tracks the ideal with a maximum deviation of 1.44%. The maximum deviation of capacitor voltage is 38 mV or 1.03%.

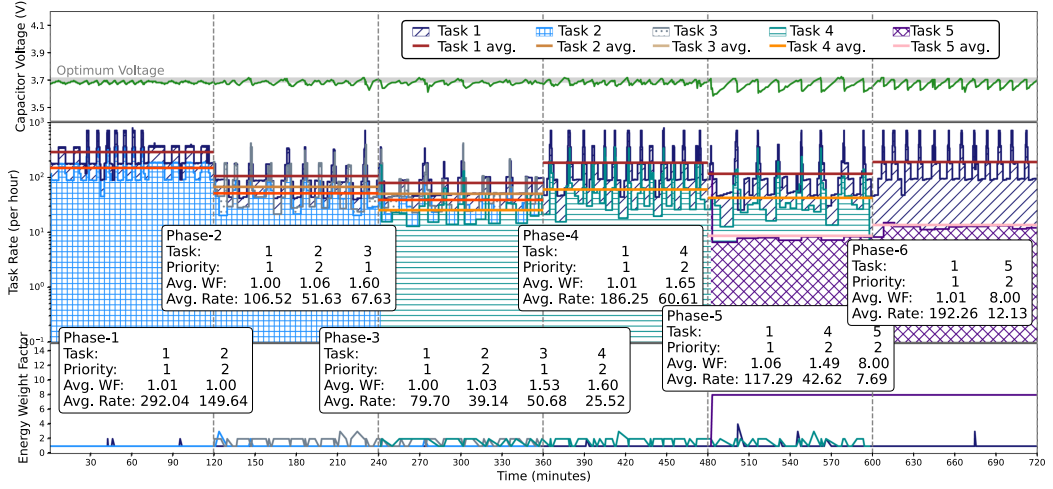


Fig. 18. Evaluation result of multiple tasks with the different load, scheduled by AsTAR with priorities and energy weight factors.

- **Phase-2:** At 120 min, Task 3 is registered with the system with a load of 20 mA and a priority of 1. AsTAR recalculates their energy weight factors to be 1.00, 1.06, and 1.60 on average, and taking into account the task priorities schedules Task 1 to 3 at a rate of 106.52, 51.63, and 67.63 executions per hour on average. The maximum deviation of capacitor voltage from the optimum is 73 mV or 1.97%. The allocation of all task schedules closely tracks the ideal with a maximum deviation of 2.68%.
- **Phase-3:** At 240 min, Task 4 is registered with the system using a load of 20 mA and a priority of 2. AsTAR recalculates the energy weight factors of Tasks 1 to 4 as 1.00, 1.03, 1.53, and 1.60, respectively, and, considering their priorities schedules them at 79.70, 39.14, 50.68, and 25.52 executions per hour on average. The maximum deviation of capacitor voltage in this period is 87 mV or 2.35% from the optimum. The allocation of all task schedules closely tracks the ideal with a maximum deviation of 5.05%.
- **Phase-4:** At 360 min, Tasks 2 and 3 are removed from the system. AsTAR recalculates the energy weight factors of Tasks 1 and 4 to be 1.01 and 1.65 on average and thus schedules them at 186.25 and 60.61 executions per hour on average. The maximum deviation of capacitor voltage in this period is 59 mV or 1.59% from the optimum. The allocation of all task schedules tracks the ideal with a maximum deviation of 5.95%.
- **Phase-5:** At 480 min, Task 5 is registered with the system, using a heavy load of 100 mA. AsTAR recalculates the energy weight factors of Tasks 1, 4, and 5 to be 1.06, 1.49, and 8.00 on average, and considering their priorities schedules them at 117.29, 42.62, and 7.69 per hour on average, the maximum deviation of capacitor voltage is 115 mV or 3.11%. The allocation of all task schedules tracks the ideal with a maximum deviation of 3.22%. As in our previous experiments, the introduction of a new task with a high energy load does not have a problematic impact on the other tasks.
- **Phase-6:** At 600 min, Task 4 is removed from the system, leaving Tasks 1 and 5. AsTAR recalculates the energy weight factors of Tasks 1 and 5 to be 1.01 and 8.00 on average, and considering their priorities schedules them at 192.26 and 12.13 per hour. The maximum deviation of capacitor voltage is 87 mV or 2.35%. The allocation of all task schedules closely tracks the ideal with a maximum deviation of 0.05%.

Table 2. Flash and RAM Memory Overhead of AsTAR

Flash for runtime	RAM for runtime	RAM for each task
4.8 kB	19 B	12 B

Table 3. Time and Charge Required for One Processing Cycle of the AsTAR Scheduler

	Time (μ s)	Charge (μ C)
Shut-off state	21.70	0.028
Low-voltage state	104.75	0.136
Optimum-voltage state	87.80	0.114
High-voltage state	121.70	0.158
Average	83.99	0.109

As shown in Figure 18, throughout the evaluation, AsTAR balances the energy consumption of tasks with developer-specified priorities, while ensuring that the energy reserve of the system remains close to the optimum voltage at all times.

5.3 Overhead of AsTAR

This section isolates the overhead of AsTAR in relation to the underlying software stack in terms of: memory, computation, and energy. As discussed in Section 3.2 and shown in Algorithm 1, the AsTAR adaptive algorithm is extremely lightweight. In each adaptation cycle, it only executes a small number of efficient instructions: *comparison*, *increment*, *decrement*, *multiply by 2* (i.e., *left shift*), *divide by 2* (*shift right*), divide and one analog read operation. As shown in Table 2, the AsTAR runtime including the algorithm and task scheduler takes only 4.8 kB of flash memory in total. Thanks to the compact structure with minimal required states, AsTAR only needs 19 Bytes of RAM for the runtime, and each task introduced into the system only needs 12 Bytes extra RAM.

Table 3 shows the time and charge required to compute adaptations in each state. As the execution period was smaller than the temporal resolution of our laboratory equipment, we executed each adaptation 10 times and provided the average. The associated energy consumption is based upon the power consumption numbers specified in the LTC5800 data sheet. The primary power cost arises from the active time of the processor, during which it has a current draw of 1.3 mA. Considered in sum, AsTAR is extremely efficient and has very limited overhead in terms of computation, energy and memory. As shown by the real-world solar experiment in Figure 14, 39.79% of time the system was in shut-off state with task execution suspended. For the remainder of the experiment tasks were operative and sending data. Specifically; 41.67% of time it was in Low-voltage state, 15.49% of time it was in Optimum state, and 3.06% of time it was in High Voltage state.

6 CONCLUSIONS AND FUTURE WORK

This article introduced AsTAR, a novel approach to building energy-harvesting Internet-of-Things applications. AsTAR embodies two core scientific contributions: (i) a simple, yet effective energy-aware task scheduling scheme for on IETF Class-1 devices, with (ii) support for task prioritization based upon either absolute importance or task energy consumption. This is supported by the engineering contribution of (iii) a reference platform for experimentation with sustainable energy-harvesting applications.

Our evaluation shows that AsTAR is capable of reliable and sustainable operation under laboratory tests and a real-world solar energy-harvesting scenario. AsTAR is a simple and robust approach to optimizing task schedules in the face of heterogeneous hardware and changing operational conditions. When combined with prioritized scheduling, it provides developers with a simple and yet expressive means to control task schedules with respect to multiple dimensions of fairness.

It is important to note that AsTAR achieves all of these benefits in a fully autonomic and self-adaptive manner with zero modeling prior to device deployment and extremely low runtime overhead. Considered as a whole, we believe that the features of AsTAR represent a significant reduction in the complexity of writing energy-aware IoT applications for energy-harvesting platforms. We note, however, that AsTAR's applicability is limited to those application-layer tasks that can be independently scheduled.

Our future work will follow two main scientific tracks. First, we will explore how AsTAR can be extended to support the scheduling of networks of cooperating local tasks, which cannot be independently throttled due to their local dependencies. Second, we plan to extend AsTAR to support the scheduling of network tasks, which impact not only the local device but also interact with tasks running on remote devices.

Complementing these scientific tracks, we are actively working on three engineering tracks. First, we will implement a hardware circuit for supporting autonomous low-power alert, avoiding the corner case when a single-task execution consuming all energy in AsTAR. Second, we will establish a large-scale AsTAR testbed that we will make available to the IoT research community to accelerate the development of energy-harvesting algorithms, tools and applications. The third track will focus upon the integration of new energy-harvesting peripherals for gathering RF, thermal and kinetic energy.

REFERENCES

- [1] Naveed Anwar Bhatti, Muhammad Hamad Alizai, Affan A. Syed, and Luca Mottola. 2016. Energy harvesting and wireless transfer in sensor network applications: Concepts and experiences. *ACM Trans. Sen. Netw.* 12, 3, Article 24 (Aug. 2016), 40 pages. <https://doi.org/10.1145/2915918>
- [2] Bernhard Buchli, Felix Sutton, Jan Beutel, and Lothar Thiele. 2014. Towards enabling uninterrupted long-term operation of solar energy harvesting embedded systems. In *Proceedings of the 11th European Conference on Wireless Sensor Networks (EWSN'14)*. Springer-Verlag, New York, NY, 66–83. https://doi.org/10.1007/978-3-319-04651-8_5
- [3] Q. Cao, D. Fesehaye, N. Pham, Y. Sarwar, and T. Abdelzaher. 2008. Virtual battery: An energy reserve abstraction for embedded sensor networks. In *Proceedings of the Real-Time Systems Symposium*. 123–133. <https://doi.org/10.1109/RTSS.2008.41>
- [4] Vinton Cerf, Yogen Dalal, and Carl Sunshine. 1974. Specification of Internet Transmission Control Program. Retrieved from <https://tools.ietf.org/html/rfc675>.
- [5] Dah-Ming Chiu and Raj Jain. 1989. Analysis of the increase and decrease algorithms for congestion avoidance in computer networks. *Comput. Netw. ISDN Syst.* 17, 1 (1989), 1–14. [https://doi.org/10.1016/0169-7552\(89\)90019-6](https://doi.org/10.1016/0169-7552(89)90019-6)
- [6] A. Dunkels, B. Gronvall, and T. Voigt. 2004. Contiki—A lightweight and flexible operating system for tiny networked sensors. In *Proceedings of the 29th Annual IEEE International Conference on Local Computer Networks*. 455–462. <https://doi.org/10.1109/LCN.2004.38>
- [7] Eaton. 2017. Technical Data 4403:PHV Supercapacitors Cylindrical pack. Retrieved from <https://www.eaton.com/content/dam/eaton/products/electronic-components/resources/data-sheet/eaton-phv-supercapacitors-cylindrical-pack-data-sheet.pdf>.
- [8] Anand Eswaran, Anthony Rowe, and Raj Rajkumar. 2005. Nano-RK: An energy-aware resource-centric RTOS for sensor networks. In *Proceedings of the 26th IEEE International Real-Time Systems Symposium (RTSS'05)*. IEEE Computer Society, Washington, DC, 256–265. <https://doi.org/10.1109/RTSS.2005.30>
- [9] Andrea Gaglione, David Rodenas-Herraiz, Yu Jia, Sarfraz Nawaz, Emmanuelle Arroyo, Cecilia Mascolo, Kenichi Soga, and Ashwin A. Seshia. 2018. Energy neutral operation of vibration energy-harvesting sensor networks for bridge applications. In *Proceedings of the International Conference on Embedded Wireless Systems and Networks (EWSN'18)*. Junction Publishing, 1–12. Retrieved from <http://dl.acm.org/citation.cfm?id=3234847.3234849>.

- [10] Josiah Hester, Lanny Sitanayah, and Jacob Sorber. 2015. Tragedy of the coulombs: Federating energy storage for tiny, intermittently-powered sensors. In *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems (SenSys'15)*. ACM, New York, NY, 5–16. <https://doi.org/10.1145/2809695.2809707>
- [11] Josiah Hester and Jacob Sorber. 2017. Flicker: Rapid prototyping for the batteryless internet-of-things. In *Proceedings of the 15th ACM Conference on Embedded Networked Sensor Systems (SenSys'17)*. ACM, New York, NY, Article 19, 13 pages. <https://doi.org/10.1145/3131672.3131674>
- [12] Jason Hsu, Sadaf Zahedi, Aman Kansal, Mani Srivastava, and Vijay Raghunathan. 2006. Adaptive duty cycling for energy harvesting systems. In *Proceedings of the International Symposium on Low Power Electronics and Design (ISLPED'06)*. ACM, New York, NY, 180–185. <https://doi.org/10.1145/1165573.1165616>
- [13] D. Hughes, P. Greenwood, G. Coulson, and G. Blair. 2006. GridStix: Supporting flood prediction using embedded hardware and next generation grid middleware. In *Proceedings of the International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM'06)*. 626. <https://doi.org/10.1109/WOWMOM.2006.49>
- [14] IETF. 2014. Terminology for Constrained-Node Networks. Retrieved from <https://tools.ietf.org/html/rfc7228#section-3>.
- [15] Keysight Technologies. B2901A Precision Source/Measure Unit, 1 ch, 100 fA, 210 V, 3 A DC/10.5 A Pulse. Retrieved from <https://www.keysight.com/en/pd-1983568-pn-B2901A/precision-source-measure-unit-1-ch-100-fa-210-v-3-a-dc-105-a-pulse?cc=BE&lc=dut>.
- [16] Andreas Lachenmann, Pedro José Marrón, Daniel Minder, and Kurt Rothermel. 2007. Meeting lifetime goals with energy levels. In *Proceedings of the 5th International Conference on Embedded Networked Sensor Systems (SenSys'07)*. ACM, New York, NY, USA, 131–144. <https://doi.org/10.1145/1322263.1322277>
- [17] Trong Nhan Le, Alain Pegatoquet, Olivier Berder, and Olivier Sentieys. 2014. A power manager with balanced quality of service for energy-harvesting wireless sensor nodes. In *Proceedings of the 2nd International Workshop on Energy Neutral Sensing Systems (ENSsys'14)*. Association for Computing Machinery, New York, NY, 1924. <https://doi.org/10.1145/2675683.2675687>
- [18] Arnott Matt, Owen Pierce, Buckland Emma, and Ranken Margaret. 2017. IoT Global Forecast and Analysis, 2015–2025. Retrieved from <https://www.gartner.com/doc/3659018/iot-global-forecast-analysis>.
- [19] N. Matthys, F. Yang, W. Daniels, S. Michiels, W. Joosen, D. Hughes, and T. Watteyne. 2015. μ PnP-Mesh: The plug-and-play mesh network for the internet of things. In *Proceedings of the IEEE 2nd World Forum on Internet of Things (WF-IoT'15)*. 311–315. <https://doi.org/10.1109/WF-IoT.2015.7389072>
- [20] Pelino Michele, S. Hammond Jeffrey, Dai Charlie, Miller Paul, Belissnt Jennifer, A. Ask Julie, Fenwick Nigel, E. Gillett Frank, Husson Thomas, Voce Merritt, Maxim with Christopher, Garberg Clare, and Lynch Diane. 2017. Predictions 2018: IoT Moves from Experimentation to Business Scale. Retrieved from <https://www.forrester.com/report/Predictions+2018+IoT+Moves+From+Experimentation+To+Business+Scale/-/E-RES139752>; <https://www.forrester.com/report/Predictions+2018+IoT+Moves+From+Experimentation+To+Business+Scale/-/E-RES139752>.
- [21] H. Ritter, J. Schiller, T. Voigt, A. Dunkels, and J. Alonso. 2005. Experimental evaluation of lifetime bounds for wireless sensor networks. In *Proceedings of the 2nd European Workshop on Wireless Sensor Networks*. 25–32. <https://doi.org/10.1109/EWSN.2005.1461996>
- [22] Arjun Roy, Stephen M. Rumble, Ryan Stutsman, Philip Levis, David Mazières, and Nickolai Zeldovich. 2011. Energy management in mobile devices with the cinder operating system. In *Proceedings of the 6th Conference on Computer Systems (EuroSys'11)*. ACM, New York, NY, 139–152. <https://doi.org/10.1145/1966445.1966459>
- [23] P. Sawyer, N. Bencomo, D. Hughes, P. Grace, H. J. Goldsby, and B. H. C. Cheng. 2007. Visualizing the analysis of dynamically adaptive systems using i^* and DSLs. In *Proceedings of the 2nd International Workshop on Requirements Engineering Visualization (REV'07)*. 3–3. <https://doi.org/10.1109/REV.2007.10>
- [24] Jacob Sorber, Alexander Kostadinov, Matthew Garber, Matthew Brennan, Mark D. Corner, and Emery D. Berger. 2007. Eon: A language and runtime system for perpetual systems. In *Proceedings of the 5th International Conference on Embedded Networked Sensor Systems (SenSys'07)*. ACM, New York, NY, 161–174. <https://doi.org/10.1145/1322263.1322279>
- [25] M. K. Stojcev, M. R. Kosanovic, and L. R. Golubovic. 2009. Power management and energy harvesting techniques for wireless sensor nodes. In *Proceedings of the 9th International Conference on Telecommunication in Modern Satellite, Cable, and Broadcasting Services*. 65–72.
- [26] Jay Taneja, Jaemin Jeong, and David Culler. 2008. Design, modeling, and capacity planning for micro-solar power sensor networks. In *Proceedings of the 7th International Conference on Information Processing in Sensor Networks (IPSN'08)*. IEEE Computer Society, Washington, DC, 407–418. <https://doi.org/10.1109/IPSN.2008.67>
- [27] Texas Instruments. 2014. INA219: 26V, 12-bit, i2c output current/voltage/power monitor. Retrieved from <https://www.ti.com/product/INA219>.
- [28] Vijay Raghunathan, A. Kansal, J. Hsu, J. Friedman, and Mani Srivastava. 2005. Design considerations for solar energy harvesting wireless embedded systems. In *Proceedings of the 4th International Symposium on Information Processing in Sensor Networks, 2005*. 457–462.

- [29] T. Voigt, H. Ritter, and J. Schiller. 2003. Utilizing solar power in wireless sensor networks. In *Proceedings of the 28th Annual IEEE International Conference on Local Computer Networks (LCN'03)*. 416–422.
- [30] T. Watteyne, L. Doherty, J. Simon, and K. Pister. 2013. Technical overview of smartmesh IP. In *Proceedings of the 7th International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing*. 547–551. <https://doi.org/10.1109/IMIS.2013.97>
- [31] Fan Yang, Nelson Matthys, Rafael Bachiller, Sam Michiels, Wouter Joosen, and Danny Hughes. 2015. μ PnP: Plug and play peripherals for the internet of things. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys'15)*. ACM, New York, NY, Article 25, 14 pages. <https://doi.org/10.1145/2741948.2741980>
- [32] Kasım Sinan Yıldırım, Amjad Yousef Majid, Dimitris Patoukas, Koen Schaper, Przemysław Pawelczak, and Josiah Hester. 2018. InK: Reactive kernel for tiny batteryless sensors. In *Proceedings of the 16th ACM Conference on Embedded Networked Sensor Systems (SenSys'18)*. ACM, New York, NY, 41–53. <https://doi.org/10.1145/3274783.3274837>
- [33] Haitao Steve Zhu, Chaoren Lin, and Yu David Liu. 2015. A programming model for sustainable software. In *Proceedings of the 37th International Conference on Software Engineering (ICSE'15)*. IEEE Press, Piscataway, NJ, 767–777. Retrieved from <http://dl.acm.org/citation.cfm?id=2818754.2818847>.

Received September 2019; revised April 2021; accepted May 2021